```
import {
  Color,
  JsonAsset,
  Material,
  Mesh,
  MeshRenderer,
  Node,
  Quat,
  Vec3,
  isValid,
  resources,
  utils,
} from 'cc';
import { AIRFRAME, WEAPONS } from './core/Data';
import { aircraftRotation, cabinPoint } from './core/Flight';
type Geometry = {
  positions: number[];
  normals: number[];
  colors: number[];
  indices: number[];
};
const RESOURCE = 'models/aircraft/cabin';
function validate(payload: unknown): Geometry {
  if (!payload || typeof payload !== 'object')
    throw Error('Expected a geometry object');
  const g = payload as Record<keyof Geometry, unknown>;
  if (
    !Array.isArray(g.positions) ||
    !Array.isArray(g.normals) ||
    !Array.isArray(g.colors) ||
    !Array.isArray(g.indices)
  )
    throw Error('positions, normals, colors and indices must be arrays');
  const { positions, normals, colors, indices } = g;
  const vertices = positions.length / 3;
  // Cocos 3.8.8 utils.createMesh writes 16-bit indices, as in World.mesh.
  if (!Number.isInteger(vertices) || vertices < 3 || vertices > 65535)
    throw Error('positions must contain 3..65535 complete XYZ vertices');
  if (normals.length !== positions.length || colors.length !== vertices * 4)
    throw Error('Expected one XYZ normal and one RGBA color per vertex');
  if (!indices.length || indices.length % 3 !== 0 || indices.length > 8000 * 3)
    throw Error('indices must contain 1..8000 complete triangles');
  for (const [name, values] of Object.entries({
    positions,
    normals,
    colors,
    indices,
  }))
    for (const value of values)
      if (
        typeof value !== 'number' ||
```

```
        !Number.isFinite(value) ||
        !Number.isFinite(Math.fround(value))
      )
        throw Error(`${name} must contain finite float32 numbers`);
  if (colors.some((v) => v < 0 || v > 1))
    throw Error('RGBA colors must be normalized to [0, 1]');
  if (indices.some((i) => !Number.isInteger(i) || i < 0 || i >= vertices))
    throw Error('indices must be integers within the vertex array');
  return { positions, normals, colors, indices };
}
/** Explicitly owned runtime model; the main World supplies the Simulation pose. */
export class AircraftModel {
  public readonly node: Node;
  public readonly ready: Promise<void>;
  public status: 'loading' | 'ready' | 'error' | 'disposed' = 'loading';
  public error = '';
  public triangleCount = 0;
  public direction: -1 | 1 = 1;
  private settleReady!: () => void;
  private mesh: Mesh | null = null;
  private material: Material | null = null;
  private renderer: MeshRenderer | null = null;
  private bay: Node;
  private weapon = 0;
  constructor(parent: Node) {
    this.node = new Node('asset:aircraft.cabin');
    this.bay = new Node('Port gun bay (metre-authored detail)');
    // The terrain sensor excludes near-field cabin geometry; muzzle transforms remain physical.
    this.bay.layer = 1 << 29;
    this.node.addChild(this.bay);
    this.bay.setScale(
      AIRFRAME.cabinScale,
      AIRFRAME.cabinScale,
      AIRFRAME.cabinScale,
    );
    this.setWeapon(0);
    // Always resolves, including failure/cancellation; callers must inspect status.
    this.ready = new Promise<void>((resolve) => {
      this.settleReady = resolve;
    });
    this.node.once(Node.EventType.NODE_DESTROYED, this.finishDisposal, this);
    if (!isValid(parent, true)) {
      this.dispose();
      return;
    }
    try {
      parent.addChild(this.node);
      if (!this.alive()) return;
      resources.load(RESOURCE, JsonAsset, (error, asset) => {
        let retained: JsonAsset | null = null;
        try {
```

```
            // Also balance late callbacks: never force-release a shared cached asset.
            if (asset) {
              asset.addRef();
              retained = asset;
            }
            if (!this.alive()) return;
            if (error) throw error;
            if (!asset) throw Error('Loader returned no JsonAsset');
            const geometry = validate(asset.json);
            this.mesh = new Mesh();
            utils.createMesh(geometry, this.mesh, { calculateBounds: true });
            this.material = new Material();
            this.material.initialize({
              effectName: 'builtin-unlit',
              defines: { USE_VERTEX_COLOR: true },
            });
            this.material.setProperty('mainColor', Color.WHITE);
            this.renderer = this.bay.addComponent(MeshRenderer);
            this.renderer.mesh = this.mesh;
            this.renderer.setMaterial(this.material, 0);
            this.triangleCount = geometry.indices.length / 3;
            this.status = 'ready';
          } catch (cause) {
            this.fail(cause);
          } finally {
            // createMesh copies into its own buffers; the JSON is no longer needed.
            retained?.decRef();
            this.settleReady();
          }
        });
      } catch (cause) {
        this.fail(cause);
      }
    }
    /** Uses Flight's shared Ry(yaw) * Rx(-pitch) * Rz(bank); direction never mirrors port/starboard. */
    public update(
      position: { x: number; y: number; z: number; bank?: number },
      yaw: number,
      pitch: number,
      direction: -1 | 1,
      bank = position.bank ?? 0,
      weapon = 0,
    ): void {
      if (!this.alive()) return;
      this.direction = direction;
      this.setWeapon(weapon);
      this.node.setPosition(position.x, position.y, position.z);
      const q = aircraftRotation({ yaw, pitch, bank });
      this.node.setRotation(new Quat(q.x, q.y, q.z, q.w));
    }
    private setWeapon(weapon: number) {
```

AircraftModel.ts:107 — AircraftModel.ts:159（排版副本行号）

```
    this.weapon = WEAPONS[weapon] ? weapon : 0;
    const origin = cabinPoint({ x: 0, y: 0, z: 0 }, this.weapon);
    this.bay.setPosition(origin.x, origin.y, origin.z);
  }
  /** Includes the parent world transform; available during loading, null after disposal. */
  public get worldMuzzle(): Vec3 | null {
    if (!this.alive()) return null;
    const mount = WEAPONS[this.weapon].muzzle;
    return Vec3.transformMat4(
      new Vec3(),
      new Vec3(mount.x, mount.y, mount.z),
      this.node.worldMatrix,
    );
  }
  public dispose(): void {
    if (this.status === 'disposed') return;
    this.node.off(Node.EventType.NODE_DESTROYED, this.finishDisposal, this);
    this.finishDisposal();
    if (isValid(this.node, true)) this.node.destroy();
  }
  private alive(): boolean {
    if (this.status === 'disposed') return false;
    // Parent/ancestor destroy() is deferred; its children may still look valid this frame.
    for (let node: Node | null = this.node; node; node = node.parent)
      if (!isValid(node, true)) {
        this.dispose();
        return false;
      }
    return true;
  }
  private fail(cause: unknown): void {
    if (this.status === 'disposed') return;
    this.status = 'error';
    this.error = cause instanceof Error ? cause.message : String(cause);
    this.releaseGraphics();
    console.error(
      `[AircraftModel] Failed to load ${RESOURCE}: ${this.error}`,
      cause,
    );
    this.settleReady();
  }
  // A node destruction event only releases ownership; do not destroy an in-flight node again.
  private finishDisposal(): void {
    if (this.status === 'disposed') return;
    this.status = 'disposed';
    this.releaseGraphics();
    this.settleReady();
  }
  private releaseGraphics(): void {
    if (this.renderer && isValid(this.renderer, true)) {
      this.renderer.mesh = null;
```

AircraftModel.ts:160 — AircraftModel.ts:216（排版副本行号）

```
01        this.renderer.setMaterial(null, 0);
02      }
03      if (this.mesh && isValid(this.mesh, true)) this.mesh.destroy();
04      if (this.material && isValid(this.material, true)) this.material.destroy();
05      this.renderer = null;
06      this.mesh = null;
07      this.material = null;
08      this.triangleCount = 0;
09    }
10  }
11  import type {} from '../../scripts/dev-mode.js';
12  import type { Overwatch } from './Overwatch';
13  import { MISSIONS, type MissionId } from './core/MissionCatalog.ts';
14  /** Package mode and server capability are independent; all asset writes use the normal journal. */
15  export function installDeveloperTools(game: Overwatch) {
16    const scope = globalThis as unknown as Window;
17    const dev = scope.SmallGamesDev;
18    const enabled = () =>
19      scope.__SMALL_GAMES_BUILD__?.type === 'development' && dev?.isEnabled();
20    if (!enabled()) return () => {};
21    const ready = () => {
22      if (!enabled()) throw Error('当前包未启用开发调试');
23      if (
24        game.sim.phase === 'playing' ||
25        game.online?.room ||
26        game.online?.active ||
27        game.online?.busy ||
28        game.quartermaster?.busy ||
29        game.quartermaster?.viewing
30      )
31        throw Error('请先正常结束当前行动、离开联机房间并等待操作完成');
32      const q = game.quartermaster;
33      if (!q) throw Error('军需服务尚未就绪');
34      return q;
35    };
36    const adjust = async (field: 'coinsDelta' | 'xpDelta', value?: number) => {
37      const q = ready();
38      if (!Number.isSafeInteger(value) || Math.abs(value!) > 1e9)
39        throw Error('请输入 ±1000000000 范围内的整数');
40      if (!q.assets?.devToolsEnabled)
41        throw Error('服务器未启用开发调试，请连接已启用开发能力的测试服');
42      if (!(await q.execute({ kind: 'dev-adjust', [field]: value })))
43        throw Error(q.message[0] || '操作未完成');
44    };
45    const select = (id: MissionId, index: number) => {
46      const q = ready();
47      q.page = '';
48      game.online?.show('');
49      game.selectMission(id);
50      game.hud.missionsOpen = id !== 'training-60';
51      game.hud.missionPage = Math.floor(index / 3);
```

AircraftModel.ts:217 — DeveloperTools.ts:42（排版副本行号）

```
      game.hud.modalKey = 'rebuild';
    };
    const stopActions = dev.registerActions([
      {
        id: 'night-coins',
        label: '调整测试金币',
        input: {
          label: '金币增减（负数扣减）',
          min: -1e9,
          max: 1e9,
          step: 1,
          value: 1000,
        },
        run: (value) => adjust('coinsDelta', value),
      },
      {
        id: 'night-xp',
        label: '调整测试经验',
        input: {
          label: '经验增减（负数扣减）',
          min: -1e9,
          max: 1e9,
          step: 1,
          value: 200,
        },
        run: (value) => adjust('xpDelta', value),
      },
      {
        id: 'night-mission',
        label: '选择试玩任务',
        input: {
          label: `任务序号（1–${MISSIONS.length}）`,
          min: 1,
          max: MISSIONS.length,
          step: 1,
          value: 1,
        },
        run(value) {
          if (!Number.isInteger(value) || value! < 1 || value! > MISSIONS.length)
            throw Error('任务序号超出范围');
          select(MISSIONS[value! - 1].id, value! - 1);
        },
      },
      {
        id: 'night-training',
        label: '选择火控热身',
        run: () => select('training-60', 0),
      },
      {
        id: 'night-assets-refresh',
        label: '刷新服务端资产',
```

```
      async run() {
        const q = ready();
        await q.refresh();
        if (q.message[0]) throw Error(q.message[0]);
      },
    },
    {
      id: 'night-assets-retry',
      label: '重试待确认操作',
      async run() {
        const q = ready();
        if (!q.pending) throw Error('没有待确认的操作');
        if (!(await q.execute())) throw Error(q.message[0] || '操作未完成');
      },
    },
  ]);
  const stopSnapshot = dev.registerSnapshot(() => game.snapshot());
  return () => {
    stopActions();
    stopSnapshot();
  };
}
import { Color, Graphics, view } from 'cc';
import { FLIGHT, WEAPONS } from './core/Data';
import type { Point } from './core/Data';
import type { Simulation } from './core/Simulation';
import type { World } from './World';
type ScreenPoint = { x: number; y: number };
export type EffectsFrame = {
  smoke?: { id: number; age: number; kind: 'damage' | 'wreck' }[];
  fires?: { id: number; age: number; life: number }[];
  lights?: number[];
  impacts: {
    id: number;
    weapon: number;
    outcome?: string;
    x: number;
    y: number;
    radius: number;
    age: number;
    primitives: number;
  }[];
  projectiles: {
    id: number;
    weapon: number;
    x: number;
    y: number;
    width: number;
    length: number;
    speed: number;
  }[];
```

DeveloperTools.ts:94 — Effects.ts:30（排版副本行号）

```
};
const TAU = Math.PI * 2;
const clamp = (n: number) => Math.max(0, Math.min(1, n));
/**
 * Append after Graphics.clear(), before target/aim markers; preserves their Graphics styles.
 * Budget: 16 rounds, 24 large / 8 small impacts, 8 attacks, 12 smoking vehicles.
 * With <=96px lobe radii and an estimated <=128 vertices/ellipse, allow ~46k FX vertices
 * including fixed-segment strokes/ribbons (Creator's default tessellation; estimate, not measured).
 */
export function drawEffects(
  g: Graphics,
  s: Simulation,
  world: World,
  width: number,
  height: number,
  reduced = false,
): EffectsFrame {
  const frame: EffectsFrame = {
    impacts: [],
    projectiles: [],
    smoke: [],
    fires: [],
    lights: [],
  };
  let primitives = 0;
  const fill = g.fillColor.clone(),
    stroke = g.strokeColor.clone(),
    lineWidth = g.lineWidth;
  const sx = view.getScaleX(),
    sy = view.getScaleY();
  const project = (p: Point, y = 0): ScreenPoint => {
    const q = world.project(p, y);
    return { x: q.x / sx - width / 2, y: q.y / sy - height / 2 };
  };
  const airborne = (p: { x: number; y: number; z: number }): ScreenPoint => {
    const q = world.projectAir(p);
    return { x: q.x / sx - width / 2, y: q.y / sy - height / 2 };
  };
  // Perspective size comes from each effect's ground location.
  const scaleAt = (p: Point) => {
    const base = project(p),
      x = project({ x: p.x + 1, z: p.z }),
      z = project({ x: p.x, z: p.z + 1 });
    // The X axis can point straight away from the camera; use both axes so the blast cannot collapse.
    return Math.min(
      30,
      Math.max(
        Math.hypot(x.x - base.x, x.y - base.y),
        Math.hypot(z.x - base.x, z.y - base.y),
      ),
    );
```

Effects.ts:31 — Effects.ts:82（排版副本行号）

```
  };
  const ink = new Color();
  const color = (heat: number, alpha: number, smoke = false): Color => {
    heat = clamp(heat);
    const gray = Math.round(60 + heat * 195);
    ink.set(
      world.thermal ? gray : smoke ? 72 + heat * 35 : 255,
      world.thermal ? gray : smoke ? 75 + heat * 32 : 95 + heat * 140,
      world.thermal ? gray : smoke ? 79 + heat * 28 : 25 + heat * 175,
      Math.round(clamp(alpha) * 255),
    );
    return ink;
  };
  const blob = (
    p: ScreenPoint,
    rx: number,
    ry: number,
    heat: number,
    alpha: number,
    smoke = false,
  ) => {
    if (
      alpha <= 0.005 ||
      rx <= 0 ||
      ry <= 0 ||
      ![p.x, p.y, rx, ry].every(Number.isFinite)
    )
      return;
    // ponytail: cap billboard lobes at 96 UI pixels; subdivide them if giant close-up clouds are needed.
    rx = Math.min(96, rx);
    ry = Math.min(96, ry);
    if (Math.abs(p.x) > width / 2 + rx || Math.abs(p.y) > height / 2 + ry)
      return;
    g.fillColor = color(heat, alpha, smoke);
    g.ellipse(p.x, p.y, rx, ry);
    g.fill();
    primitives++;
  };
  const line = (
    a: ScreenPoint,
    b: ScreenPoint,
    heat: number,
    alpha: number,
    weight: number,
    friendly = false,
  ) => {
    if (![a.x, a.y, b.x, b.y].every(Number.isFinite)) return;
    if (
      Math.max(a.x, b.x) < -width / 2 ||
      Math.min(a.x, b.x) > width / 2 ||
      Math.max(a.y, b.y) < -height / 2 ||
```

Effects.ts:83 — Effects.ts:133（排版副本行号）

```
        Math.min(a.y, b.y) > height / 2
      )
        return;
      g.strokeColor = color(heat, alpha);
      if (friendly && !world.thermal)
        g.strokeColor = new Color(145, 230, 203, Math.round(alpha * 255));
      g.lineWidth = weight;
      g.moveTo(a.x, a.y);
      g.lineTo(b.x, b.y);
      g.stroke();
      primitives++;
    };
    const ring = (
      p: Point,
      radius: number,
      heat: number,
      alpha: number,
      dashed = false,
    ) => {
      if (alpha <= 0.005) return;
      const q = project(p),
        x = project({ x: p.x + radius, z: p.z });
      const z = project({ x: p.x, z: p.z + radius });
      const extent = Math.max(
        Math.hypot(x.x - q.x, x.y - q.y),
        Math.hypot(z.x - q.x, z.y - q.y),
      );
      if (
        Math.abs(q.x) > width / 2 + extent ||
        Math.abs(q.y) > height / 2 + extent ||
        extent > 2000
      )
        return;
      g.strokeColor = color(heat, alpha);
      g.lineWidth = 1;
      // Project both ground axes, including temporary zoom; 24 segments also bound stroke geometry.
      for (let i = 0; i < (dashed ? 24 : 25); i++) {
        const a = (i * TAU) / 24;
        const px = q.x + Math.cos(a) * (x.x - q.x) + Math.sin(a) * (z.x - q.x);
        const py = q.y + Math.cos(a) * (x.y - q.y) + Math.sin(a) * (z.y - q.y);
        if (i === 0 || (dashed && i % 2 === 0)) g.moveTo(px, py);
        else g.lineTo(px, py);
      }
      if (!dashed) g.close();
      g.stroke();
      primitives++;
    };
    const patch = (points: ScreenPoint[], tint: Color) => {
      if (
        points.some(
          (p) =>
```

```
            !Number.isFinite(p.x + p.y) ||
            Math.abs(p.x) > width * 3 ||
            Math.abs(p.y) > height * 3,
        )
      )
        return;
      g.fillColor = tint;
      points.forEach((p, i) => (i ? g.lineTo(p.x, p.y) : g.moveTo(p.x, p.y)));
      g.close();
      g.fill();
    };
    for (const zone of s.fireZones) {
      const age = s.time - zone.born,
        left = zone.expires - s.time;
      if (age < 0 || left <= 0) continue;
      const fade = Math.min(1, left / 2),
        scale = scaleAt(zone);
      patch(
        Array.from({ length: 24 }, (_, i) => {
          const angle = (i * TAU) / 24;
          return project(
            {
              x: zone.x + Math.cos(angle) * zone.radius,
              z: zone.z + Math.sin(angle) * zone.radius,
            },
            0.06,
          );
        }),
        world.thermal
          ? new Color(235, 235, 235, 45 * fade)
          : new Color(201, 69, 19, 80 * fade),
      );
      ring(zone, zone.radius, 0.45, 0.9 * fade);
      // Connected sheets keep the burning area readable without a field of isolated sparks.
      for (let band = 0; band < (reduced ? 2 : 3); band++) {
        const z = zone.z + (band - 1) * zone.radius * 0.52;
        const span = Math.sqrt(zone.radius ** 2 - (z - zone.z) ** 2) * 0.95;
        const base = [],
          crest = [];
        for (let i = 0; i <= 16; i++) {
          const t = i / 16,
            x = zone.x + (t * 2 - 1) * span;
          const edge = Math.sin(t * Math.PI),
            wave =
              0.65 +
              0.35 * Math.sin(t * Math.PI * 8 + age * (reduced ? 1.8 : 3) + band);
          base.push(project({ x, z: z + zone.radius * 0.32 }, 0.08));
          crest.push(project({ x, z }, 0.15 + edge * (1.5 + wave * 2)));
        }
        patch(
          [...base, ...crest.reverse()],
```

Effects.ts:186 — Effects.ts:236（排版副本行号）

```
          world.thermal
            ? new Color(245, 245, 235, 165 * fade)
            : new Color(255, 94 + band * 37, 24, (155 + band * 20) * fade),
        );
      }
      frame.fires!.push({ id: zone.id, age, life: zone.expires - zone.born });
    }
    for (const u of s.units) {
      const at = project(u);
      if (Math.abs(at.x) > width / 2 + 40 || Math.abs(at.y) > height / 2 + 40)
        continue;
      const radius = u.kind === 'heavy' ? 0.5 : 0.35;
      for (let layer = 0; layer < 3; layer++) {
        const r = radius + layer * 0.14;
        patch(
          Array.from({ length: 12 }, (_, j) => {
            const a = (j * TAU) / 12;
            return project(
              {
                x: u.x + 0.3 + Math.cos(a) * r,
                z: u.z + 0.25 + Math.sin(a) * r * 1.6,
              },
              0.012 + layer * 0.004,
            );
          }),
          new Color(3, 8, 12, world.thermal ? 12 : 28),
        );
      }
      if (
        u.hp <= 0 ||
        u.kind === 'turret' ||
        world.thermal ||
        world.light === 'day'
      )
        continue;
      const c = Math.cos(u.heading),
        sn = Math.sin(u.heading);
      const point = (side: number, forward: number, y = 0.025) =>
        project(
          {
            x: u.x + c * side + sn * forward,
            z: u.z - sn * side + c * forward,
          },
          y,
        );
      for (const side of [-0.16, 0.16]) {
        patch(
          [
            point(side - 0.06, 0.45),
            point(side - 0.75, 5.5),
            point(side + 0.75, 5.5),
```

Effects.ts:237 — Effects.ts:287（排版副本行号）

```
            point(side + 0.06, 0.45),
          ],
          new Color(255, 222, 157, reduced ? 13 : 22),
        );
        patch(
          [
            point(side - 0.04, 0.45),
            point(side - 0.35, 2.7),
            point(side + 0.35, 2.7),
            point(side + 0.04, 0.45),
          ],
          new Color(255, 229, 171, 34),
        );
        const p = point(side, 0.48, 0.23);
        blob(p, 2.7, 2.3, 0.75, 0.18);
        blob(p, 1.1, 1, 1, 0.95);
      }
      frame.lights!.push(u.id);
    }
    // ponytail: inspect 64 units and render 12 plumes; spatial pooling if missions grow.
    let wrecks = 0;
    for (
      let i = s.units.length - 1, end = Math.max(0, s.units.length - 64);
      i >= end && wrecks < 12;
      i--
    ) {
      const u = s.units[i],
        dead = u.hp <= 0,
        age = dead ? s.time - u.deadAt : s.time;
      if (dead ? u.deadAt < 0 || age < 0 || age >= 35 : u.hp / u.maxHp > 0.55)
        continue;
      wrecks++;
      const scale = scaleAt(u);
      const fade = dead
        ? Math.min(1, age * 2) * Math.min(1, (35 - age) / 10)
        : 0.6;
      for (let j = reduced ? 0 : 3; j >= 0; j--) {
        const drift = (age * 0.35 + j * 0.7) % 3;
        const r = (0.16 + drift * 0.14) * scale;
        const p = project(
          { x: u.x + drift * 0.25 + Math.sin(u.id + j) * 0.08, z: u.z },
          0.3 + drift * 0.6,
        );
        blob(
          p,
          Math.max(1.5, r),
          Math.max(2, r * 1.2),
          dead ? 0.6 * Math.max(0, 1 - age / 30) : 0.3,
          fade * (1 - drift / 3) * 0.6,
          true,
        );
```

Effects.ts:288 — Effects.ts:339（排版副本行号）

```
    }
    frame.smoke!.push({ id: u.id, age, kind: dead ? 'wreck' : 'damage' });
    if (dead && age < 18) {
      const p = project(u, 0.3),
        flicker = 0.75 + Math.sin(age * 13 + u.id) * 0.2;
      blob(
        p,
        Math.max(1.5, scale * 0.2),
        Math.max(3, scale * 0.5 * flicker),
        0.85,
        (1 - age / 18) * 0.9,
      );
      frame.fires!.push({ id: u.id, age, life: 18 });
    }
  }
  // Special weapons remain presentation-only; authoritative hits come from the simulation.
  for (const beam of s.lasers) {
    const unit = s.units.find((unit) => unit.id === beam.targetId);
    if (!unit || unit.hp <= 0) continue;
    const origin = airborne(beam.origin),
      target = airborne(unit),
      pulse = 0.82 + Math.sin(s.time * 36) * 0.12;
    if (!reduced) line(origin, target, 0.7, 0.25 * pulse, 7, true);
    line(origin, target, 1, pulse, reduced ? 1.5 : 2.5, true);
    blob(target, 5, 5, 1, pulse);
  }
  for (const drone of s.drones.slice(0, 6)) {
    const p = airborne(drone),
      size = Math.max(4, Math.min(12, scaleAt(drone) * 0.45)),
      falling = s.time >= drone.expires;
    const angle = falling ? (s.time - drone.expires) * 9 : 0,
      cos = Math.cos(angle),
      sin = Math.sin(angle);
    const rotor = (x: number, y: number) => ({
      x: p.x + x * cos - y * sin,
      y: p.y + x * sin + y * cos,
    });
    for (const sign of [-1, 1])
      line(
        rotor(-size, sign * size * 0.45),
        rotor(size, -sign * size * 0.45),
        0.8,
        0.9,
        2,
        true,
      );
    for (const dx of [-1, 1])
      for (const dy of [-1, 1]) {
        const r = rotor(dx * size, dy * size * 0.45);
        g.strokeColor = falling
          ? new Color(245, 168, 91, 210)
```

Effects.ts:340 — Effects.ts:391（排版副本行号）

```
            : new Color(143, 222, 230, 210);
          g.lineWidth = 1;
          g.ellipse(r.x, r.y, size * 0.45, size * 0.15);
          g.stroke();
        }
      blob(p, 2, 2, 1, 0.9);
      if (falling)
        for (let i = 0; i < (reduced ? 1 : 3); i++)
          blob(
            { x: p.x + i * 2, y: p.y + size + i * 5 },
            3 + i,
            4 + i,
            0.25,
            0.5 - i * 0.12,
            true,
          );
      const target = s.units.find((u) => u.id === drone.target);
      if (
        !falling &&
        target &&
        s.time > drone.nextAttack - 0.12 &&
        s.time <= drone.nextAttack
      )
        line(p, project(target, target.y + 0.2), 1, 0.65, 1, true);
    }
    for (const unit of s.units)
      if ((unit.rootedUntil ?? 0) > s.time) {
        const p = project(unit, unit.y + 0.5),
          r = Math.max(5, Math.min(30, scaleAt(unit) * 0.8));
        g.strokeColor = new Color(112, 213, 255, 210);
        g.lineWidth = 1.5;
        g.ellipse(p.x, p.y, r, r * 0.45);
        g.stroke();
        if (!reduced) {
          g.moveTo(p.x - r * 0.3, p.y + r);
          g.lineTo(p.x + r * 0.2, p.y + r * 0.4);
          g.lineTo(p.x - r * 0.15, p.y);
          g.lineTo(p.x + r * 0.3, p.y - r * 0.6);
          g.stroke();
        }
      }
    // Every terrain impact explodes, including misses. Kill events use wreck smoke, not a second explosion.
    let attacks = 0;
    for (const e of [
      ...world.impactClouds,
      ...s.events.filter((event) => event.type === 'attack'),
    ]) {
      const age = s.time - e.time;
      if (age < 0) continue;
      if (e.type === 'attack') {
        if (age >= 0.3 || attacks++ >= 8) continue;
```

Effects.ts:392 — Effects.ts:442（排版副本行号）

```
      const target = e.target || s.units.find((u) => u.id === e.unit);
      if (!target) continue;
      const a = project(e, 0.3),
        b = project(target, 0.25),
        t = clamp(age / 0.12);
      line(a, b, 0.45, (1 - age / 0.3) * 0.55, 1, e.friendly);
      blob(
        { x: a.x + (b.x - a.x) * t, y: a.y + (b.y - a.y) * t },
        2,
        2,
        1,
        1 - age / 0.3,
      );
      continue;
    }
    if (e.type !== 'impact') continue;
    const weapon = e.weapon,
      life = weapon === 0 ? 1.5 : weapon === 1 ? 10 : 16;
    if (age >= life) continue;
    const before = primitives;
    const scale = scaleAt(e);
    const radius = WEAPONS[weapon].radius;
    const burnLife =
      weapon === 0
        ? 0.35
        : e.outcome === 'destroyed'
          ? 8
          : weapon === 2
            ? 5
            : 2.8;
    const t = age / life,
      fade = 1 - t,
      seed = (e.id * 0.61803398875) % 1;
    const wave = clamp(age / (weapon === 0 ? 0.2 : 0.75));
    // The pressure ring stops at the gameplay blast radius; dust can drift beyond it.
    ring(e, radius * (0.2 + wave * 0.8), 0.9 - t * 0.65, (1 - wave) * 0.8);
    if (weapon > 0) {
      ring(e, radius * (0.12 + wave * 0.88), 0.5, (1 - wave) * 0.4);
      if (reduced) {
        const r = Math.max(1.5, radius * (0.15 + t * 0.3) * scale);
        blob(
          project(e, radius * 0.15),
          r,
          r * 0.6,
          fade * 0.5,
          fade * 0.45,
          true,
        );
      }
      const lobes = reduced ? 0 : weapon === 2 ? 5 : 3;
      // Cool outer dust behind fewer, smaller fire lobes. All motion is a function of event id and age.
```

Effects.ts:443 — Effects.ts:493（排版副本行号）

```
        for (let j = lobes - 1; j >= 0; j--) {
          const a = seed * TAU + j * 2.39996323;
          const spread = radius * (0.1 + t * 0.55);
          const p = project(
            {
              x: e.x + Math.cos(a) * spread + age * 0.12,
              z: e.z + Math.sin(a) * spread,
            },
            radius * (0.08 + t * (0.65 + j * 0.08)),
          );
          const r = radius * (0.16 + t * 0.42 + (j % 2) * 0.05) * scale;
          blob(p, r, r * 1.12, fade * 0.55, fade * 0.48, true);
        }
        for (let j = 0; j < (reduced ? 0 : weapon === 2 ? 3 : 2); j++) {
          const a = seed * TAU + (j * TAU) / 3,
            burn = clamp(1 - age / burnLife);
          const p = project(
            {
              x: e.x + Math.cos(a) * radius * 0.2,
              z: e.z + Math.sin(a) * radius * 0.2,
            },
            radius * (0.1 + t * 0.7),
          );
          const r =
            radius *
            (0.16 + Math.sin(Math.min(1, t * 2) * Math.PI) * 0.2) *
            scale;
          blob(p, r, r * 1.1, burn, burn * 0.78);
        }
      } else {
        const p = project(e, 0.2);
        blob(p, (0.35 + age) * scale, (0.25 + age) * scale, fade, fade * 0.45);
      }
      if (weapon > 0 && age < burnLife) {
        const fire =
          (1 - age / burnLife) * (0.8 + Math.sin(age * 17 + e.id) * 0.15);
        const p = project(e, radius * 0.08);
        blob(
          p,
          Math.max(3, radius * 0.3 * scale),
          Math.max(2, radius * 0.13 * scale),
          0.65,
          fire * 0.24,
        );
        blob(
          p,
          Math.max(1.5, radius * 0.12 * scale),
          Math.max(3, radius * 0.28 * scale),
          0.95,
          fire * 0.9,
        );
```

Effects.ts:494 — Effects.ts:544（排版副本行号）

```
        frame.fires!.push({ id: e.id, age, life: burnLife });
      }
      const flash = clamp(1 - age / (weapon === 2 ? 0.24 : 0.16));
      const core = Math.max(weapon === 0 ? 1 : 2, radius * 0.3 * scale);
      blob(project(e, 0.25), core, core, 1, flash * 0.9);
      const count = reduced ? 1 : weapon === 0 ? 3 : weapon === 1 ? 5 : 7;
      if (age < (weapon === 0 ? 0.28 : 0.85)) {
        for (let j = 0; j < count; j++) {
          const a = seed * TAU + j * 2.39996323;
          const speed = (1.7 + ((e.id + j * 13) % 7) * 0.22) * radius;
          const fragment = (time: number) =>
            project(
              {
                x: e.x + Math.cos(a) * speed * time,
                z: e.z + Math.sin(a) * speed * time,
              },
              Math.max(0, 0.15 + radius * 1.7 * time - 5 * time * time),
            );
          line(
            fragment(Math.max(0, age - 0.045)),
            fragment(age),
            fade,
            clamp(1 - age / (weapon === 0 ? 0.28 : 0.85)) * 0.9,
            weapon === 2 ? 1.6 : 1,
          );
        }
      }
      if (primitives > before) {
        const p = project(e);
        frame.impacts.push({
          id: e.id,
          weapon,
          outcome: e.outcome,
          x: p.x + width / 2,
          y: height / 2 - p.y,
          radius: radius * scale,
          age,
          primitives: primitives - before,
        });
      }
    }
    // ponytail: last 16 rounds cover current cadence; raise this budget if weapons gain simultaneous volleys.
    for (let i = Math.max(0, s.shots.length - 16); i < s.shots.length; i++) {
      const shot = s.shots[i],
        age = s.time - shot.born,
        flight = shot.due - shot.born;
      if (flight <= 0 || age < 0 || age >= flight) continue;
      const t = clamp(age / flight),
        weapon = shot.weapon,
        spec = WEAPONS[weapon],
```

Effects.ts:545 — Effects.ts:595（排版副本行号）

```
      missile = shot.guidance?.kind === 'consumable';
    const position = (progress: number): ScreenPoint =>
      airborne(s.shotPosition(shot, shot.born + progress * flight));
    if (!missile)
      ring(
        shot,
        WEAPONS[weapon].radius * (1 + (1 - t) * 0.65),
        0.75,
        0.22 + t * 0.25,
        true,
      );
    // A single tapered ribbon per round, seven samples along the actual origin-to-impact parabola.
    const tail: ScreenPoint[] = [];
    const start = Math.max(0, t - (missile ? 0.45 : spec.tracerTime) / flight);
    for (let j = 0; j <= 6; j++)
      tail.push(position(start + ((t - start) * j) / 6));
    if (
      tail.some(
        (p) =>
          !Number.isFinite(p.x) ||
          !Number.isFinite(p.y) ||
          Math.abs(p.x) > width * 3 ||
          Math.abs(p.y) > height * 3,
      )
    )
      continue;
    const location = s.shotPosition(shot),
      projected = airborne(location);
    const localScale = Math.min(
      200,
      Math.max(
        ...[
          { x: location.x + 1, y: location.y, z: location.z },
          { x: location.x, y: location.y + 1, z: location.z },
          { x: location.x, y: location.y, z: location.z + 1 },
        ].map((p) => {
          const q = airborne(p);
          return Math.hypot(q.x - projected.x, q.y - projected.y);
        }),
      ),
    );
    // Keep distant rounds readable in UI pixels without changing their ballistic path.
    const bodyWidth = missile
      ? 5
      : Math.min(5, Math.max(1.5 + weapon * 0.65, spec.calibre * localScale));
    const bodyLength = missile
      ? 12
      : Math.min(12, Math.max(4 + weapon * 1.5, spec.length * localScale));
    g.fillColor = color(0.8, 0.7);
    for (let side = 0; side < 2; side++) {
```

Effects.ts:596 — Effects.ts:645（排版副本行号）

```
        for (let n = 0; n <= 6; n++) {
          const j = side === 0 ? n : 6 - n,
            p = tail[j];
          const a = tail[Math.max(0, j - 1)],
            b = tail[Math.min(6, j + 1)];
          const length = Math.max(0.001, Math.hypot(b.x - a.x, b.y - a.y));
          const w = (0.08 + (j / 6) * bodyWidth * 0.5) * (side === 0 ? 1 : -1);
          const x = p.x - ((b.y - a.y) / length) * w,
            y = p.y + ((b.x - a.x) / length) * w;
          if (side === 0 && n === 0) g.moveTo(x, y);
          else g.lineTo(x, y);
        }
      }
      g.close();
      g.fill();
      const head = tail[6],
        last = tail[5],
        heading = Math.atan2(head.y - last.y, head.x - last.x);
      const beforeHead = primitives;
      const body = (along: number, across: number) => ({
        x: head.x + Math.cos(heading) * along - Math.sin(heading) * across,
        y: head.y + Math.sin(heading) * along + Math.cos(heading) * across,
      });
      // A pointed nose and tapered tail remain readable even when a round is only a few pixels long.
      patch(
        [
          body(1, 0),
          body(-bodyLength * 0.32, bodyWidth * 0.5),
          body(-bodyLength * 0.82, bodyWidth * 0.5),
          body(-bodyLength, bodyWidth * 0.24),
          body(-bodyLength, -bodyWidth * 0.24),
          body(-bodyLength * 0.82, -bodyWidth * 0.5),
          body(-bodyLength * 0.32, -bodyWidth * 0.5),
        ],
        color(0.85, 0.98).clone(),
      );
      primitives++;
      if (missile)
        for (const side of [-1, 1])
          patch(
            [
              body(-bodyLength * 0.6, side * bodyWidth * 0.3),
              body(-bodyLength, side * bodyWidth),
              body(-bodyLength * 0.9, 0),
            ],
            color(0.5, 0.9).clone(),
          );
      line(
        body(-bodyLength * 0.75, -bodyWidth * 0.12),
        body(-bodyLength * 0.25, -bodyWidth * 0.12),
```

Effects.ts:646 — Effects.ts:695（排版副本行号）

```
      1,
      0.8,
      Math.max(0.6, bodyWidth * 0.2),
    );
    if (primitives > beforeHead)
      frame.projectiles.push({
        id: shot.id,
        weapon,
        x: head.x + width / 2,
        y: height / 2 - head.y,
        width: bodyWidth,
        length: bodyLength,
        speed:
          (shot.timeScale ?? 1) *
          Math.hypot(
            shot.velocity.x,
            shot.velocity.y -
              (shot.guidance
                ? 0
                : FLIGHT.gravity * age * (shot.timeScale ?? 1)),
            shot.velocity.z,
          ),
      });
    const flash = shot.timeScale ? 0 : clamp(1 - age / 0.085);
    if (flash > 0) {
      const origin = airborne(shot.origin);
      blob(origin, 7 + weapon * 3, 5 + weapon * 2, 0.65, flash * 0.35);
      blob(origin, 2.5 + weapon, 2.5 + weapon, 1, flash);
    }
  }
  g.fillColor = fill;
  g.strokeColor = stroke;
  g.lineWidth = lineWidth;
  return frame;
}
import { safeArea, watchSafeArea, fitName } from './core/SafeArea';
import { playerId, playerLabel, type Online } from './Online';
import {
  Camera,
  Canvas,
  Color,
  Graphics,
  Label,
  Layers,
  Mask,
  Node,
  UITransform,
  view,
  ResolutionPolicy,
  sys,
```

Effects.ts:696 — HUD.ts:15（排版副本行号）

```
  Sprite,
  SpriteFrame,
  Texture2D,
  Rect,
  resources,
  isValid,
  EditBox,
  UIOpacity,
  tween,
  Tween,
} from 'cc';
import {
  ACTIONS,
  actionLabel,
  bindingLabel,
  TUTORIAL,
  tutorialText,
} from './core/Actions';
import { drawEffects, type EffectsFrame } from './Effects';
import {
  WEAPONS,
  MAP,
  FLIGHT,
  terrainHeight,
  ROUTE,
  HOLD_POINTS,
  FRIENDLY_POSTS,
  routePoint,
  PROTECTED,
  text,
  type Language,
  type Point,
} from './core/Data';
import type { Simulation } from './core/Simulation';
import type { World } from './World';
import {
  missionUnlocked,
  newMissionProgress,
  type MissionProgress,
} from './core/MissionProgress';
import { MISSIONS, TRAINING, type MissionLight } from './core/MissionCatalog';
import { mapRoute, routeLength, riverX, type BattlefieldId } from './core/Data';
import { trainingNextGoal, type TrainingRecord } from './core/TrainingRecords';
import {
  nextZoomLimit,
  type RewardKind,
  type SupplyReward,
} from './core/Rewards';
import {
  battleCounts,
```

HUD.ts:16 — HUD.ts:65（排版副本行号）

```
  formatTime,
  type TimeComparison,
} from './core/BattleRecords';
import { ProgressionArt } from './ProgressionArt';
import { ProgressionView } from './ProgressionView';
import { CATALOG, SKILLS, attackPreview } from './core/Progression';
import { effectEnglish } from './Quartermaster';
import type { Quartermaster } from './Quartermaster';
const C = {
  ink: '#081725f5',
  line: '#456780',
  white: '#e5edf3',
  dim: '#9bb4c8',
  mint: '#82c9ed',
  amber: '#e5b65c',
  action: '#ffe3a4',
  red: '#ff525b',
};
const col = (hex: string) => new Color().fromHEX(hex);
const movableControls = [
  'weapon0',
  'weapon1',
  'weapon2',
  'weapon3',
  'homing',
  'weapon4',
  'skillUse',
  'weapon5',
];
type ControlPosition = { x: number; y: number };
export type ButtonRect = {
  id: string;
  x: number;
  y: number;
  w: number;
  h: number;
  label: Label;
  visual?: { x: number; y: number; w: number; h: number };
  disabled?: boolean;
  clip?: { x: number; y: number; w: number; h: number };
};
export class HUD {
  listAreas: Array<{
    key: string;
    x: number;
    y: number;
    w: number;
    h: number;
    max: number;
    axis: 'x' | 'y';
```

HUD.ts:66 — HUD.ts:116（排版副本行号）

```
  }> = [];
  listScroll: Record<string, number> = {};
  missionFocus: number | undefined;
  safeRevision = 0;
  private nameFits = new Map<string, string>();
  nativeCapsule = false;
  private semanticKey = '';
  listAt(x: number, y: number) {
    return [...this.listAreas]
      .reverse()
      .find((a) => x >= a.x && x <= a.x + a.w && y >= a.y && y <= a.y + a.h);
  }
  moveList(key: string, amount: number) {
    const a = this.listAreas.find((a) => a.key === key);
    if (!a) return;
    const states = key.startsWith('qm:')
      ? this.quartermaster!.scrolls
      : this.listScroll;
    states[key] = Math.max(0, Math.min(a.max, (states[key] || 0) + amount));
    this.modalKey = 'rebuild';
  }
  listViewport(
    parent: Node,
    key: string,
    area: { x: number; y: number; w: number; h: number },
    total: number,
    axis: 'x' | 'y' = 'y',
  ) {
    const max = Math.max(0, total - (axis === 'x' ? area.w : area.h)),
      states = key.startsWith('qm:')
        ? this.quartermaster!.scrolls
        : this.listScroll;
    const offset = (states[key] = Math.max(0, Math.min(max, states[key] || 0)));
    this.listAreas.push({ ...area, key, max, axis });
    const viewport = this.node('Scroll:' + key, parent),
      mx = area.x + area.w / 2 - this.w / 2,
      my = this.h / 2 - area.y - area.h / 2;
    viewport.setPosition(mx, my);
    viewport.getComponent(UITransform)!.setContentSize(area.w, area.h);
    viewport.addComponent(Mask).type = Mask.Type.GRAPHICS_RECT;
    const n = this.node('ScrollContent', viewport);
    n.setPosition(-mx, -my);
    const surfaces = this.node('Surfaces', n),
      g = this.node('Ink', n).addComponent(Graphics),
      start = this.buttons.length;
    return {
      n,
      surfaces,
      g,
      offset,
```

HUD.ts:117 — HUD.ts:166（排版副本行号）

```
01        finish: () => {
02          for (const b of this.buttons.slice(start)) b.clip = area;
03          if (max && axis === 'x') {
04            const ink = this.node('ScrollIndicator', parent).addComponent(
05                Graphics,
06              ),
07              track = area.w - 32,
08              thumb = Math.max(20, (track * area.w) / total),
09              y = area.y + area.h - 5;
10            this.rect(ink, area.x + 16, y, track, 3, '#253e50');
11            this.rect(
12              ink,
13              area.x + 16 + ((track - thumb) * offset) / max,
14              y - 1,
15              thumb,
16              5,
17              C.amber,
18            );
19            if (offset > 1)
20              this.label(
21                parent,
22                '‹',
23                area.x + 5,
24                area.y + area.h / 2,
25                25,
26                14,
27                32,
28                C.white,
29              );
30            if (offset < max - 1)
31              this.label(
32                parent,
33                '›',
34                area.x + area.w - 5,
35                area.y + area.h / 2,
36                25,
37                14,
38                32,
39                C.white,
40              );
41          }
42          if (max && axis === 'y') {
43            const ink = this.node('ScrollIndicator', parent).addComponent(
44                Graphics,
45              ),
46              x = area.x + area.w - 5,
47              track = Math.max(4, area.h - 32),
48              thumb = Math.max(20, (track * area.h) / total);
49            this.rect(ink, x, area.y + 16, 3, track, '#253e50');
50            this.rect(
```

HUD.ts:167 — HUD.ts:216（排版副本行号）

```
            ink,
            x - 1,
            area.y + 16 + ((track - thumb) * offset) / max,
            5,
            thumb,
            C.amber,
          );
          const arrow = (y: number, down: boolean) => {
            ink.strokeColor = col(C.white);
            ink.lineWidth = 2;
            const xx = x - this.w / 2,
              yy = this.h / 2 - y,
              d = down ? -1 : 1;
            ink.moveTo(xx - 4, yy - d * 2);
            ink.lineTo(xx, yy + d * 2);
            ink.lineTo(xx + 4, yy - d * 2);
            ink.stroke();
          };
          if (offset > 1) arrow(area.y + 6, false);
          if (offset < max - 1) arrow(area.y + area.h - 7, true);
        }
      },
    };
  }
  quartermaster?: Quartermaster;
  progressionArt: ProgressionArt;
  private progressionRevision = '';
  root: Node;
  camera: Camera;
  g: Graphics;
  marks: Graphics;
  globalControls: Node;
  globalGraphics: Graphics;
  homeBackground: Node;
  homeArtwork: Sprite;
  private titleArtwork?: SpriteFrame;
  private missionArtwork: SpriteFrame[] = [];
  private dayArtwork: SpriteFrame[] = [];
  private artworkFrames: SpriteFrame[] = [];
  w = 960;
  h = 540;
  lang: Language = 'zh';
  radioText = '';
  touch =
    sys.isMobile ||
    (sys.isBrowser && window.matchMedia('(pointer: coarse)').matches);
  // UI coordinates from real mouse input; the controller clears this on leave/touch/blur.
  mousePointer?: { x: number; y: number };
  pressedButton = '';
  private uiFrames = new Map<string, SpriteFrame>();
```

HUD.ts:217 — HUD.ts:266（排版副本行号）

```
  private artworkRetries = new Map<string, () => void>();
  private surfaces?: Node;
  retryArtwork() {
    this.progressionArt.retry();
    for (const [name, retry] of Array.from(this.artworkRetries)) {
      this.artworkRetries.delete(name);
      retry();
    }
    this.modalKey = 'rebuild';
  }
  get interfaceArtworkReady() {
    return this.quartermaster
      ? this.progressionArt.homeReady
      : this.uiFrames.size === 3 &&
          !!this.titleArtwork &&
          this.missionArtwork.length === 3 &&
          this.dayArtwork.length === 3;
  }
  effects: EffectsFrame = { impacts: [], projectiles: [] };
  buttons: ButtonRect[] = [];
  labels = new Map<string, Label>();
  modal?: Node;
  modalKey = '';
  private onlineRevision = -1;
  private modalFamily = '';
  screenControl = false;
  screenActive = false;
  missionsOpen = false;
  online?: Online;
  reportOpen = false;
  equipmentOpen = false;
  timeComparison?: TimeComparison;
  missionPage = 0;
  missionProgress: MissionProgress = newMissionProgress();
  developer = false;
  helpGroup: 'basic' | 'advanced' | 'rules' = 'basic';
  helpTouch = false;
  scroll = 0;
  scrollMax = 0;
  scrollNode?: Node;
  scrollArea = { x: 0, y: 0, w: 0, h: 0 };
  tutorial = true;
  muted = false;
  voicePrompts = true;
  autoPriority = true;
  reducedEffects = false;
  toolsOpen = false;
  zoomOpen = false;
  zoomLimit = 10;
  rewardsAvailable = false;
```

HUD.ts:267 — HUD.ts:316（排版副本行号）

```
  supplyOpen = false;
  exitConfirm = false;
  supplySelection: SupplyReward = 'tracking';
  supplyMessage = '';
  pendingSupply = false;
  advert?: { kind: RewardKind; mock: boolean };
  lockedTargets: { id: number; x: number; y: number }[] = [];
  toast = '';
  toastUntil = 0;
  trainingBest?: TrainingRecord;
  unitLabels = new Map<number, Label>();
  lastHelpLine = '';
  safe = { left: 0, right: 0, top: 0, bottom: 0 };
  controlPositions: Record<string, ControlPosition> = {};
  controlBounds(id: string) {
    const b = this.buttons.find((b) => b.id === id)!;
    return {
      x: this.safe.left,
      y: this.safe.top,
      w: Math.max(0, this.w - this.safe.left - this.safe.right - b.w),
      h: Math.max(0, this.h - this.safe.top - this.safe.bottom - b.h),
    };
  }
  private positionControl(b: ButtonRect, x: number, y: number, width = b.w) {
    b.x = x;
    b.y = y;
    b.w = width;
    b.visual = { x: x + 2, y: y + 2, w: width - 4, h: b.h - 4 };
    b.label.node.setPosition(
      x + width / 2 - this.w / 2,
      this.h / 2 - y - b.h / 2,
    );
    b.label.node.getComponent(UITransform)!.setContentSize(width - 8, b.h - 2);
    b.label.overflow = Label.Overflow.SHRINK;
  }
  placeControl(id: string, position?: ControlPosition) {
    if (position)
      this.controlPositions[id] = {
        x: Math.max(0, Math.min(1, position.x)),
        y: Math.max(0, Math.min(1, position.y)),
      };
    else delete this.controlPositions[id];
    this.layoutCombatControls();
  }
  controlAt(x: number, y: number) {
    const button = this.hit(x, y, true);
    return button && movableControls.includes(button.id) ? button : undefined;
  }
  private avoidFixedControls(id: string) {
    const b = this.buttons.find((b) => b.id === id)!,
```

HUD.ts:317 — HUD.ts:366（排版副本行号）

```
        bounds = this.controlBounds(id);
      if (!b.label.node.activeInHierarchy) return;
      for (const fixed of this.buttons
        .filter(
          (b) =>
            !movableControls.includes(b.id) &&
            !b.disabled &&
            b.label.node.activeInHierarchy,
        )
        .sort((a, b) => b.x - a.x)) {
        if (
          b.x >= fixed.x + fixed.w ||
          b.x + b.w <= fixed.x ||
          b.y >= fixed.y + fixed.h ||
          b.y + b.h <= fixed.y
        )
          continue;
        const above = fixed.y > this.h / 2 && fixed.y - b.h - 4 >= bounds.y;
        const x =
          !above && fixed.x - b.w - 4 >= bounds.x ? fixed.x - b.w - 4 : b.x;
        const y = above
          ? fixed.y - b.h - 4
          : x !== b.x
            ? b.y
            : Math.min(bounds.y + bounds.h, fixed.y + fixed.h + 4);
        this.positionControl(b, x, y);
        this.controlPositions[id] = {
          x: (x - bounds.x) / Math.max(1, bounds.w),
          y: (y - bounds.y) / Math.max(1, bounds.h),
        };
      }
    }
    private layoutCombatControls() {
      const left = 12 + this.safe.left,
        right = this.w - 12 - this.safe.right,
        by = this.h - this.footer - this.safe.bottom;
      const normal = this.buttons.filter((b) =>
        movableControls.slice(0, 5).includes(b.id),
      );
      const support = ['weapon4', 'skillUse', 'weapon5']
        .map((id) => this.buttons.find((b) => b.id === id)!)
        .filter((b) => b.label.node.activeInHierarchy);
      const visible = normal.filter((b) => b.label.node.activeInHierarchy),
        fire = this.buttons.find((b) => b.id === 'fire')!;
      const cardW = Math.min(
        this.compact ? 64 : 78,
        (right - left - 146) / 5 - 6,
      );
      let regularW = cardW,
        supportW = 100;
```

HUD.ts:367 — HUD.ts:416（排版副本行号）

```
    if (support.length) {
      const room = fire.x - left - 12,
        gaps =
          Math.max(0, visible.length - 1) * 6 + 8 + (support.length - 1) * 6;
      supportW = Math.min(
        100,
        Math.max(72, (room - visible.length * cardW - gaps) / support.length),
      );
      regularW = Math.max(
        44,
        Math.min(
          cardW,
          (room - support.length * supportW - gaps) /
            Math.max(1, visible.length),
        ),
      );
    }
    normal.forEach((b, index) =>
      this.positionControl(b, left + index * (cardW + 6), by + 8, cardW),
    );
    if (support.length)
      visible.forEach((b, index) =>
        this.positionControl(
          b,
          left + index * (regularW + 6),
          by + 8,
          regularW,
        ),
      );
    const dockX = left + visible.length * (regularW + 6) + 2;
    support.forEach((b, index) =>
      this.positionControl(b, dockX + index * (supportW + 6), by + 8, supportW),
    );
    const state = this.labels.get('weaponState')!,
      stateW = Math.max(
        60,
        Math.min(132, right - 68 - (left + 5 * (cardW + 6)) - 12),
      );
    const crowded =
      support.length > 0 &&
      dockX + support.length * (supportW + 6) - 6 > fire.x - stateW - 16;
    state.node.setPosition(
      (crowded ? right - stateW / 2 : fire.x - 8 - stateW / 2) - this.w / 2,
      this.h / 2 -
        (crowded ? fire.y - 8 - (this.footer - 14) / 2 : fire.y + fire.h / 2),
    );
    state.node
      .getComponent(UITransform)!
      .setContentSize(stateW, this.footer - 14);
    for (const id of movableControls) {
```

HUD.ts:417 — HUD.ts:466（排版副本行号）

完整源程序第 520 页 共 549 页

```
const cutoff = Date.parse('2026-11-01T00:00:00+08:00');
const fail = (code: string): never => {
  throw new CompetitionError(code, 422);
};
const emptyLoadout = (): Loadout => ({ mods: [], weapon: null, skill: null });
const integer = (value: unknown) =>
  typeof value === 'number' && Number.isSafeInteger(value) && value >= 0
    ? value
    : 0;
const canonical = (v: unknown): unknown =>
  Array.isArray(v)
    ? v.map(canonical)
    : v && typeof v === 'object'
      ? Object.fromEntries(
          Object.entries(v)
            .sort(([a], [b]) => a.localeCompare(b))
            .map(([k, x]) => [k, canonical(x)]),
        )
      : v;
const signature = (v: unknown) =>
  createHash('sha256')
    .update(JSON.stringify(canonical(v)))
    .digest('hex');
const ACHIEVEMENTS = [
  {
    id: 'sorties-10',
    name: '十次出动',
    event: 'complete',
    target: 10,
    reward: { coins: 300, xp: 150, packs: 2 },
  },
  {
    id: 'victories-25',
    name: '常胜守望',
    event: 'win',
    target: 25,
    reward: { coins: 500, xp: 300, permanent: 'tracking' },
  },
  {
    id: 'sorties-100',
    name: '百战守望',
    event: 'complete',
    target: 100,
    reward: { coins: 1000, xp: 500, permanent: 'emp' },
  },
];
const templates = [
  {
    id: 'checkin',
    name: '完成今日签到',
```

night-assets.ts:220 — night-assets.ts:269（排版副本行号）

```
      event: 'checkin',
      target: 1,
      reward: { coins: 60, xp: 40 },
    },
    {
      id: 'solo',
      name: '完成 2 次单人任务',
      event: 'solo',
      target: 2,
      reward: { coins: 120, xp: 80 },
    },
    {
      id: 'duel',
      name: '完成 2 次双人对战',
      event: 'duel',
      target: 2,
      reward: { coins: 120, xp: 80 },
    },
    {
      id: 'open',
      name: '打开 1 个礼包',
      event: 'open',
      target: 1,
      reward: { coins: 40, xp: 30 },
    },
    {
      id: 'complete',
      name: '完成 3 次有效行动',
      event: 'complete',
      target: 3,
      reward: { coins: 140, xp: 90 },
    },
    {
      id: 'win',
      name: '赢得 1 次行动',
      event: 'win',
      target: 1,
      reward: { coins: 100, xp: 70 },
    },
  ];
  function prepare(
    w: NightAssets,
    owner: string,
    at: number,
    rules: ProgressionRules,
  ) {
    const day = nightDay(at),
      oldest = nightDay(at - 7 * 86400000);
    w.tasks = w.tasks.filter((t) => t.day >= oldest);
    if (!w.tasks.some((t) => t.day === day)) {
```

```
    const seed = createHash('sha256')
      .update(`${owner}:${day}`)
      .digest()
      .readUInt32LE();
    const selected = [
      templates[0]!,
      templates[1]!,
      templates[3]!,
      templates[2 + (seed % 3 === 0 ? 0 : (seed % 3) + 1)]!,
    ];
    w.tasks.push(
      ...selected.map((t) => ({
        ...structuredClone(t),
        id: `${day}:${t.id}`,
        day,
        progress: 0,
        claimed: false,
      })),
    );
  }
  for (const a of ACHIEVEMENTS)
    w.achievements[a.id] ??= { progress: 0, claimed: false };
  const level = rules.levelProgress(w.xp).level;
  for (const [effect, skill] of Object.entries(rules.skills))
    if (level >= skill.level && !w.permanent.includes(effect))
      w.permanent.push(effect);
  for (const reservation of Object.values(w.reservations))
    if (reservation.status === 'reserved' && reservation.expiresAt <= at)
      reservation.status = 'released';
}
function add(w: NightAssets, reward: Reward) {
  w.coins += reward.coins;
  w.xp += reward.xp;
  if (reward.packs) stock(w, 'pack:general:0', reward.packs);
  if (reward.permanent && !w.permanent.includes(reward.permanent))
    w.permanent.push(reward.permanent);
  if (!Number.isSafeInteger(w.coins) || !Number.isSafeInteger(w.xp))
    fail('ASSET_LIMIT');
}
function stock(w: NightAssets, id: string, n: number) {
  const value = (w.inventory[id] ?? 0) + n;
  if (!Number.isSafeInteger(value) || value < 0) fail('INSUFFICIENT_INVENTORY');
  w.inventory[id] = value;
}
function reserved(w: NightAssets, id: string) {
  return Object.values(w.reservations)
    .filter((r) => r.status === 'reserved')
    .reduce((n, r) => n + r.items.filter((i) => i === id).length, 0);
}
function available(w: NightAssets, id: string) {
```

完整源程序第 523 页 共 549 页

```
    return (w.inventory[id] ?? 0) - reserved(w, id);
  }
  function progress(w: NightAssets, day: string, events: string[], count = 1) {
    for (const task of w.tasks)
      if (task.day === day && events.includes(task.event))
        task.progress = Math.min(task.target, task.progress + count);
    for (const a of ACHIEVEMENTS)
      if (events.includes(a.event))
        w.achievements[a.id]!.progress = Math.min(
          a.target,
          w.achievements[a.id]!.progress + count,
        );
  }
  /** Read-only legacy compensation preview; legacy JSON and its ledger are never deleted. */
  export async function previewNightMigration(tx: Tx | Database, owner: string) {
    const old = await tx.execute(
      sql`select data from runtime.competition_matches where id=${`night-wallet:${owner}`}`,
    );
    const legacy = old[0]?.data as Record<string, unknown> | undefined;
    const result =
      await tx.execute(sql`select count(distinct match_id)::int as count from runtime.competition_results
      where player_id=${owner} and board like 'night-overwatch:%' and score>0 and created_at<${cutoff}`);
    const players = await tx.execute(
      sql`select created_at from runtime.competition_players where id=${owner}`,
    );
    const historical = Number(result[0]?.count ?? 0),
      veteran = !!legacy || Number(players[0]?.created_at ?? Infinity) < cutoff;
    const legacyPrices: Record<string, number[]> =
      legacy?.version === 2
        ? {
            power: [0, 900],
            velocity: [0, 600],
            duration: [0, 600],
            hePower: [300, 600, 1200],
            heVelocity: [300, 600, 1200],
            heavyPower: [300, 600, 1200],
            heavyVelocity: [300, 600, 1200],
            spread: [300, 600, 1200],
            emp: [7900, 10000, 14000],
          }
        : {
            power: [0, 900],
            velocity: [0, 600],
            duration: [0, 600],
            spread: [300, 750],
            emp: [1200],
          };
    const checkins = await tx.execute(
      sql`select created_at from runtime.competition_matches where board='night-overwatch:private-wallet-v1' and
id like ${`night-wallet-event:${owner}:request:%`} and data->>'signature'='checkin'`,
    );
```

night-assets.ts:370 — night-assets.ts:419（排版副本行号）

```
  const signedDays = [
    ...new Set(
      [
        ...(Array.isArray(legacy?.signedDays) ? legacy.signedDays : []),
        legacy?.signed,
        ...checkins.map((row) => nightDay(Number(row.created_at))),
      ].filter(
        (value): value is string =>
          typeof value === 'string' && /^\d{4}-\d{2}-\d{2}$/.test(value),
      ),
    ),
  ].sort();
  let conversionCoins = 0;
  for (const [id, tier] of Object.entries(
    (legacy?.owned as Record<string, unknown>) ?? {},
  ))
    conversionCoins += (legacyPrices[id] ?? [])
      .slice(0, Math.min(3, integer(tier)))
      .reduce((n, p) => n + p, 0);
  for (const [key, count] of Object.entries(
    (legacy?.trials as Record<string, unknown>) ?? {},
  )) {
    const [id, tier] = key.split(':');
    conversionCoins +=
      (legacyPrices[id!]?.[Number(tier) - 1] ?? 0) * integer(count);
  }
  return {
    version: 'night-return-2026-11-01',
    cutoff,
    legacy: legacy ? structuredClone(legacy) : undefined,
    preservedCoins: integer(legacy?.coins),
    veteran,
    historical,
    conversionCoins,
    signedDays,
    reward: {
      coins:
        (veteran ? 200 + Math.min(10000, historical * 60) : 200) +
        conversionCoins,
      xp: veteran ? Math.min(6000, historical * 40) : 0,
      packs: veteran ? 3 : 1,
    },
  };
}
async function locked(
  tx: Tx,
  owner: string,
  at: number,
  rules: ProgressionRules,
) {
```

night-assets.ts:420 — night-assets.ts:469（排版副本行号）

```
    // Asset lock never acquires another player's identity lock, including two-player settlement.
    await tx.execute(
      sql`select pg_advisory_xact_lock(hashtextextended(${walletId(owner)},0))`,
    );
    const existing = await tx.execute(
      sql`select data from runtime.competition_matches where id=${walletId(owner)} for update`,
    );
    let w = existing[0]?.data as NightAssets | undefined;
    if (!w) {
      const migration = await previewNightMigration(tx, owner),
        legacy = migration.legacy;
      w = {
        version: 3,
        coins: migration.preservedCoins,
        xp: 0,
        revision: 0,
        inventory: {},
        permanent: [],
        active: {},
        loadouts: { solo: emptyLoadout(), duel: emptyLoadout() },
        signedDays: migration.signedDays,
        tasks: [],
        achievements: {},
        mail: [
          {
            id: migration.version,
            title: migration.veteran ? '老玩家补偿' : '欢迎加入夜航',
            body: `历史积分、进度与累计签到已保留。旧军需转换为新体系资格，按原版本目录价值补为
${migration.conversionCoins} 金币；原始资产记录保留。附件只可领取一次。`,
            createdAt: at,
            read: false,
            claimed: false,
            reward: migration.reward,
          },
        ],
        videoOrders: [],
        videoDays: {},
        reservations: {},
        legacy,
        migratedAt: at,
      };
      if (legacy) {
        const inventory = legacy.inventory as Record<string, unknown> | undefined;
        for (const [id, count] of Object.entries(inventory ?? {}))
          if (integer(count)) {
            w.inventory[`legacy:${id}`] = integer(count);
            if (id === 'coolant') w.inventory['cooling-match'] = integer(count);
          }
        const owned = legacy.owned as Record<string, unknown> | undefined;
        for (const [id, count] of Object.entries(owned ?? {}))
          if (integer(count)) {
```

night-assets.ts:470 — night-assets.ts:519（排版副本行号）

```
            const effect = ['power', 'hePower', 'heavyPower'].includes(id)
              ? 'power'
              : ['velocity', 'heVelocity', 'heavyVelocity'].includes(id)
                ? 'rate'
                : id === 'spread' || id === 'duration'
                  ? 'radius'
                  : id === 'emp'
                    ? 'emp'
                    : null;
            if (effect && !w.permanent.includes(effect)) w.permanent.push(effect);
          }
        for (const [id, count] of Object.entries(
          (legacy.trials as Record<string, unknown>) ?? {},
        ))
          if (integer(count)) w.inventory[`legacy-trial:${id}`] = integer(count);
      }
      await tx.execute(
        sql`insert into runtime.competition_matches(id,board,data,created_at)
values(${walletId(owner)},${board},${JSON.stringify(w)}::jsonb,${at})`,
      );
    }
    if (w.version !== 3) fail('ASSET_VERSION');
    prepare(w, owner, at, rules);
    return w;
  }
  async function save(tx: Tx, owner: string, w: NightAssets) {
    w.revision++;
    await tx.execute(
      sql`update runtime.competition_matches set data=${JSON.stringify(w)}::jsonb where id=${walletId(owner)}`,
    );
  }
  const publicOrder = (order: VideoOrder) => ({
    ...(order.purpose ? { purpose: order.purpose } : {}),
    id: order.id,
    quantity: order.quantity,
    required: order.required,
    completed: order.completed,
    delivered: order.delivered,
    version: order.version,
    createdAt: order.createdAt,
    ...(order.attempt ? { attempt: order.attempt } : {}),
  });
  function snapshot(
    w: NightAssets,
    owner: string,
    at: number,
    rules: ProgressionRules,
  ) {
    const inventory: Record<string, number> = {},
      free: Record<string, number> = {};
    for (const [id, n] of Object.entries(w.inventory)) {
```

night-assets.ts:520 — night-assets.ts:569（排版副本行号）

完整源程序第 527 页 共 549 页

```
    const canonical = id.replace(/^pack:gold:/, 'pack:general:');
    inventory[canonical] = (inventory[canonical] ?? 0) + n;
    free[canonical] = (free[canonical] ?? 0) + available(w, id);
  }
  return {
    playerId: owner,
    version: w.version,
    coins: w.coins,
    xp: w.xp,
    ...rules.levelProgress(w.xp),
    revision: w.revision,
    inventory,
    available: free,
    permanent: w.permanent,
    active: w.active,
    loadouts: w.loadouts,
    tasks: w.tasks,
    achievements: ACHIEVEMENTS.map((a) => ({ ...a, ...w.achievements[a.id] })),
    mail: w.mail,
    announcements: [
      {
        id: 'growth-v1',
        title: '成长与军需上线',
        body: '积分保持原规则。金币、经验独立记录；单人和双人分别配装。技能按冷却重复使用。',
        createdAt: cutoff,
      },
    ],
    videoOrders: w.videoOrders.map(publicOrder),
    signedDays: w.signedDays,
    signedToday: w.signedDays.includes(nightDay(at)),
    day: nightDay(at),
    serverNow: at,
    videoDailyLimit: 12,
    videosToday: w.videoDays[nightDay(at)] ?? 0,
    catalog: rules.catalog,
    packages: rules.packages,
    pools: rules.pools,
    quoteVersion: rules.version,
    adVerification: 'limited-client-callback',
    legacyPreserved: !!w.legacy,
  };
}
function drawPacks(rules: ProgressionRules, pool: string, quantity: number) {
  const weights = rules.pools[pool]?.weights ?? fail('INVALID_POOL');
  const grants: Record<string, number> = {};
  for (let n = 0; n < quantity; n++) {
    let roll = randomInt(weights.reduce((a, b) => a + b, 0)),
      tier = 0;
    while (tier < weights.length - 1 && roll >= weights[tier]!)
      roll -= weights[tier++]!;
```

night-assets.ts:570 — night-assets.ts:619（排版副本行号）

```
      const id = `pack:${pool}:${tier}`;
      grants[id] = (grants[id] ?? 0) + 1;
    }
    return grants;
  }
  function deliverPacks(w: NightAssets, grants: Record<string, number>) {
    for (const [id, count] of Object.entries(grants)) stock(w, id, count);
    return grants;
  }
  function spend(w: NightAssets, coins: number) {
    if (w.coins < coins) fail('INSUFFICIENT_COINS');
    w.coins -= coins;
  }
  export async function nightAssets(
    tx: Tx,
    owner: string,
    at: number,
    rules: ProgressionRules,
    command?: AssetCommand,
    development = false,
  ) {
    if (command?.kind === 'dev-adjust' && !development)
      throw new CompetitionError('DEV_TOOLS_DISABLED', 403);
    const w = await locked(tx, owner, at, rules),
      day = nightDay(at);
    const response = async (lastReceipt?: unknown) => {
      const rows = await tx.execute(
        sql`select data,created_at from runtime.competition_matches where board=${board} and id like
${eventId(owner, 'request:%')} and data->'result'->>'kind'='open' order by created_at desc,id desc limit 20`,
      );
      const openReceipts = rows.map((row) => {
        const result = (
          row.data as {
            result: {
              requestId: string;
              item?: string;
              quantity?: number;
              drops: Array<{ item?: string; coins?: number }>;
            };
          }
        ).result;
        return {
          requestId: result.requestId,
          at: Number(row.created_at),
          item: result.item ?? 'pack',
          quantity: result.quantity ?? result.drops.length,
          drops: result.drops,
        };
      });
      return {
        ...snapshot(w, owner, at, rules),
```

night-assets.ts:620 — night-assets.ts:669（排版副本行号）

```
        devToolsEnabled: development,
        openReceipts,
        wheelTickets: w.wheelTickets ?? 0,
        wheelVideosToday: w.wheelVideoDays?.[day] ?? 0,
        wheelDailyLimit: 5,
        wheelPrizes: rules.wheelPrizes ?? [],
        ...(lastReceipt ? { lastReceipt } : {}),
      };
    };
    if (!command) {
      await save(tx, owner, w);
      return response();
    }
    const cmd = nightAssetCommandSchema.parse(command),
      key = eventId(owner, `request:${cmd.requestId}`),
      hash = signature(cmd);
    const previous = await tx.execute(
      sql`select data from runtime.competition_matches where id=${key}`,
    );
    if (previous[0]) {
      const receipt = previous[0].data as { hash: string; result: unknown };
      if (receipt.hash !== hash)
        throw new CompetitionError('REQUEST_ID_CONFLICT');
      return response(receipt.result);
    }
    if (cmd.kind === 'ad-complete') {
      const completed = await tx.execute(
        sql`select data from runtime.competition_matches where id=${eventId(owner, `ad:${cmd.attemptId}`)}`,
      );
      if (completed[0]) {
        const saved = completed[0].data as {
          orderId: string;
          result: Record<string, unknown>;
        };
        if (saved.orderId !== cmd.orderId) fail('INVALID_AD_ATTEMPT');
        const result = { ...saved.result, requestId: cmd.requestId };
        await save(tx, owner, w);
        await tx.execute(
          sql`insert into runtime.competition_matches(id,board,data,created_at)
values(${key},${board},${JSON.stringify({ hash, result, at })}::jsonb,${at})`,
        );
        return response(result);
      }
    }
    let result: Record<string, unknown> = {
      kind: cmd.kind,
      requestId: cmd.requestId,
    };
    if (cmd.kind === 'dev-adjust') {
      const before = { coins: w.coins, xp: w.xp };
      const after = { coins: w.coins + cmd.coinsDelta, xp: w.xp + cmd.xpDelta };
```

night-assets.ts:670 — night-assets.ts:719（排版副本行号）

```
    if (
      !Object.values(after).every(
        (value) => Number.isSafeInteger(value) && value >= 0,
      )
    )
      fail('INVALID_DEV_BALANCE');
    w.coins = after.coins;
    w.xp = after.xp;
    result = {
      ...result,
      before,
      after,
      coinsDelta: cmd.coinsDelta,
      xpDelta: cmd.xpDelta,
    };
  } else if (cmd.kind === 'buy') {
    const p =
      rules.catalog.find((p) => p.id === cmd.item) ?? fail('ITEM_UNAVAILABLE');
    if (
      rules.levelProgress(w.xp).level <
      (p.unlockLevel ?? rules.skills[p.effect]?.level ?? 1)
    )
      fail('LEVEL_LOCKED');
    if (
      p.term === 'permanent' &&
      (w.permanent.includes(p.effect) || cmd.quantity !== 1)
    )
      fail('ALREADY_OWNED');
    spend(w, p.price * cmd.quantity);
    if (p.term === 'permanent') w.permanent.push(p.effect);
    else stock(w, p.id, cmd.quantity);
    result = {
      ...result,
      item: p.id,
      quantity: cmd.quantity,
      coins: -p.price * cmd.quantity,
    };
  } else if (cmd.kind === 'checkin') {
    if (w.signedDays.includes(day)) fail('ALREADY_CLAIMED');
    w.signedDays.push(day);
    const reward = {
      coins: 100 + (w.signedDays.length % 7 === 0 ? 100 : 0),
      xp: 40,
      packs: 1,
    };
    add(w, reward);
    progress(w, day, ['checkin']);
    result = { ...result, reward };
  } else if (cmd.kind === 'buy-packs' || cmd.kind === 'ad-order') {
    const quote =
```

night-assets.ts:720 — night-assets.ts:769（排版副本行号）

```
        rules.packages.find((p) => p.quantity === cmd.quantity) ??
        fail('INVALID_PACKAGE');
      if (cmd.kind === 'buy-packs') {
        spend(w, quote.coins);
        result = {
          ...result,
          coins: -quote.coins,
          packs: deliverPacks(w, drawPacks(rules, 'general', cmd.quantity)),
        };
      } else {
        if (cmd.purpose === 'wheel' && cmd.quantity !== 1)
          fail('INVALID_PACKAGE');
        const wheel = cmd.purpose === 'wheel';
        if (
          w.videoOrders.filter(
            (o) => !o.delivered && (o.purpose === 'wheel') === wheel,
          ).length >= (wheel ? 1 : 5)
        )
          fail('TOO_MANY_ORDERS');
        const order: VideoOrder = {
          ...(cmd.purpose === 'wheel' ? { purpose: 'wheel' as const } : {}),
          id: randomUUID(),
          quantity: quote.quantity,
          required: cmd.purpose === 'wheel' ? 1 : quote.videos,
          completed: 0,
          delivered: false,
          version: rules.version,
          createdAt: at,
          ...(cmd.purpose === 'wheel'
            ? {}
            : { grants: drawPacks(rules, 'general', quote.quantity) }),
        };
        w.videoOrders.push(order);
        result = { ...result, order: publicOrder(order) };
      }
    } else if (cmd.kind === 'ad-cancel') {
      const order =
        w.videoOrders.find((o) => o.id === cmd.orderId) ??
        fail('ORDER_NOT_FOUND');
      if (order.attempt?.id === cmd.attemptId) delete order.attempt;
    } else if (cmd.kind === 'ad-start' || cmd.kind === 'ad-complete') {
      const order =
        w.videoOrders.find((o) => o.id === cmd.orderId) ??
        fail('ORDER_NOT_FOUND');
      if (order.delivered) fail('ALREADY_CLAIMED');
      if ((w.videoDays[day] ?? 0) >= 12) fail('DAILY_VIDEO_LIMIT');
      if (order.purpose === 'wheel' && (w.wheelVideoDays?.[day] ?? 0) >= 5)
        fail('DAILY_WHEEL_LIMIT');
      if (cmd.kind === 'ad-start') {
        const playing = w.videoOrders.find(
```

night-assets.ts:770 — night-assets.ts:819（排版副本行号）

```
          (o) =>
            !o.delivered && o.attempt && at - o.attempt.startedAt < 30 * 60000,
        );
        if (playing) fail('AD_IN_PROGRESS');
        order.attempt = { id: randomUUID(), startedAt: at };
        result = { ...result, orderId: order.id, attemptId: order.attempt!.id };
      } else {
        // shortcut: H5 exposes no server-verifiable receipt here; quota/order/time checks limit abuse, not
prove viewing.
        if (
          !order.attempt ||
          order.attempt.id !== cmd.attemptId ||
          at < order.attempt.startedAt ||
          at - order.attempt.startedAt > 30 * 60000
        )
          fail('INVALID_AD_ATTEMPT');
        if (at - order.attempt!.startedAt < 10000)
          await recordNightAdAudit(tx, owner, at, 'ad-complete', 'FAST_CALLBACK');
        order.completed++;
        w.videoDays[day] = (w.videoDays[day] ?? 0) + 1;
        if (order.purpose === 'wheel') {
          w.wheelVideoDays ??= {};
          w.wheelVideoDays[day] = (w.wheelVideoDays[day] ?? 0) + 1;
        }
        delete order.attempt;
        progress(w, day, ['video']);
        if (order.completed === order.required) {
          order.delivered = true;
          if (order.purpose === 'wheel') {
            w.wheelTickets = (w.wheelTickets ?? 0) + 1;
            result.tickets = 1;
          } else
            result.packs = deliverPacks(
              w,
              order.grants ?? drawPacks(rules, 'general', order.quantity),
            );
          progress(w, day, ['exchange']);
        }
        result = { ...result, order: publicOrder(order) };
        await tx.execute(
          sql`insert into runtime.competition_matches(id,board,data,created_at) values(${eventId(owner,
`ad:${cmd.attemptId}`)},${board},${JSON.stringify({ orderId: order.id, result, at })}::jsonb,${at})`,
        );
      }
    } else if (cmd.kind === 'wheel-spin') {
      if (!(w.wheelTickets ?? 0)) fail('NO_WHEEL_TICKETS');
      const prizes = rules.wheelPrizes;
      if (!prizes?.length) fail('ITEM_UNAVAILABLE');
      const roll = randomInt(prizes!.reduce((sum, p) => sum + p.weight, 0));
      let cumulative = 0;
      const prizeIndex = prizes!.findIndex(
          (p) => (cumulative += p.weight) > roll,
```

night-assets.ts:820 — night-assets.ts:869（排版副本行号）

```
      ),
      prize = prizes![prizeIndex]!;
    w.wheelTickets = (w.wheelTickets ?? 0) - 1;
    if (prize.coins) w.coins += prize.coins;
    const packs =
      prize.tier >= 0
        ? deliverPacks(w, { [`pack:general:${prize.tier}`]: 1 })
        : undefined;
    result = {
      ...result,
      prizeIndex,
      coins: prize.coins,
      ...(packs ? { packs } : {}),
    };
  } else if (cmd.kind === 'open' || cmd.kind === 'sell') {
    const p = rules.catalog.find((p) => p.id === cmd.item),
      pack = /^pack:(general|gold):([0-3])$/.exec(cmd.item);
    const keys = pack
      ? [`pack:general:${pack[2]}`, `pack:gold:${pack[2]}`]
      : [cmd.item];
    if (keys.reduce((sum, id) => sum + available(w, id), 0) < cmd.quantity)
      fail('INSUFFICIENT_INVENTORY');
    const consume = () => {
      let left = cmd.quantity;
      for (const id of keys) {
        const take = Math.min(left, available(w, id));
        stock(w, id, -take);
        left -= take;
      }
    };
    if (cmd.kind === 'sell') {
      const unit = pack
        ? rules.pools[pack[1]!]!.packSell
        : (p?.sell ?? (cmd.item.startsWith('legacy') ? 20 : 0));
      if (!unit) fail('ITEM_NOT_SELLABLE');
      consume();
      w.coins += unit * cmd.quantity;
      result = { ...result, coins: unit * cmd.quantity };
    } else {
      if (!pack) throw new CompetitionError('ITEM_NOT_PACK', 422);
      consume();
      const pool = rules.pools[pack[1]!]!,
        tier = Number(pack[2]);
      const drops: Array<{ item?: string; coins?: number }> = [];
      const choices = rules.catalog.filter(
        (p) =>
          p.term === 'match' &&
          p.rarity === tier &&
          p.category !== 'skill' &&
          pool.effects.includes(p.effect) &&
```

night-assets.ts:870 — night-assets.ts:919（排版副本行号）

```
            (p.unlockLevel ?? 1) <= rules.levelProgress(w.xp).level &&
            !w.permanent.includes(p.effect),
        );
        for (let n = 0; n < cmd.quantity; n++) {
          if (!choices.length) {
            const coins = pool.coinDrops[tier]!;
            w.coins += coins;
            drops.push({ coins });
          } else {
            const prize = choices[randomInt(choices.length)]!;
            stock(w, prize.id, 1);
            drops.push({ item: prize.id });
          }
        }
        progress(w, day, ['open'], cmd.quantity);
        result = { ...result, item: cmd.item, quantity: cmd.quantity, drops };
      }
    } else if (cmd.kind === 'activate') {
      const p =
        rules.catalog.find((p) => p.id === cmd.item) ?? fail('ITEM_UNAVAILABLE');
      if (w.permanent.includes(p.effect)) fail('ALREADY_OWNED');
      if (p.term !== '24h' || available(w, p.id) < 1)
        fail('INSUFFICIENT_INVENTORY');
      stock(w, p.id, -1);
      w.active[p.effect] = Math.max(at, w.active[p.effect] ?? 0) + 86400000;
    } else if (cmd.kind === 'loadout') {
      if (!rules.validLoadout(cmd.loadout)) fail('INVALID_LOADOUT');
      for (const id of [
        ...cmd.loadout.mods,
        ...[cmd.loadout.weapon, cmd.loadout.skill].filter(
          (x): x is string => !!x,
        ),
      ]) {
        const p = rules.catalog.find((p) => p.id === id)!;
        if (
          !w.permanent.includes(p.effect) &&
          !(w.active[p.effect]! > at) &&
          !(available(w, id) > 0)
        )
          fail('INSUFFICIENT_INVENTORY');
      }
      w.loadouts[cmd.mode] = structuredClone(cmd.loadout);
    } else if (
      cmd.kind === 'claim-task' ||
      cmd.kind === 'claim-all' ||
      cmd.kind === 'alternate-task'
    ) {
      const tasks =
        cmd.kind === 'claim-all'
          ? w.tasks
```

night-assets.ts:920 — night-assets.ts:969（排版副本行号）

```
          : w.tasks.filter((t) => t.id === cmd.taskId);
      if (!tasks.length) fail('TASK_NOT_FOUND');
      if (cmd.kind === 'alternate-task') {
        const t = tasks[0]!;
        if (t.event !== 'duel' || t.claimed || t.alternate)
          fail('TASK_NOT_REPLACEABLE');
        t.alternate = true;
        t.event = 'complete';
        t.name = '完成 3 次有效行动';
        t.target = 3;
        t.progress = 0;
      } else {
        let claimed = 0;
        for (const t of tasks)
          if (!t.claimed && t.progress >= t.target) {
            t.claimed = true;
            add(w, t.reward);
            claimed++;
          }
        if (!claimed) fail('NOT_CLAIMABLE');
        result.claimed = claimed;
      }
    } else if (cmd.kind === 'read-mail' || cmd.kind === 'claim-mail') {
      const mail =
        w.mail.find((m) => m.id === cmd.mailId) ?? fail('MAIL_NOT_FOUND');
      mail.read = true;
      if (cmd.kind === 'claim-mail') {
        if (mail.claimed) fail('ALREADY_CLAIMED');
        mail.claimed = true;
        add(w, mail.reward);
        result.reward = mail.reward;
      }
    } else {
      const a =
          ACHIEVEMENTS.find((a) => a.id === cmd.achievementId) ??
          fail('ACHIEVEMENT_NOT_FOUND'),
        state = w.achievements[a.id]!;
      if (state.claimed || state.progress < a.target) fail('NOT_CLAIMABLE');
      state.claimed = true;
      add(w, a.reward);
      result.reward = a.reward;
    }
    prepare(w, owner, at, rules);
    await save(tx, owner, w);
    await tx.execute(
      sql`insert into runtime.competition_matches(id,board,data,created_at)
values(${key},${board},${JSON.stringify({ hash, result, at })}::jsonb,${at})`,
    );
    return response(result);
  }
  /** Daily bounded counters contain no token, nickname, IP or raw client payload. */
```

night-assets.ts:970 — night-assets.ts:1020（排版副本行号）

```
01  export async function recordNightAdAudit(
02    tx: Tx,
03    owner: string,
04    at: number,
05    kind: string,
06    reason: string,
07  ) {
08    if (
09      !['ad-order', 'ad-start', 'ad-complete'].includes(kind) ||
10      ![
11        'FAST_CALLBACK',
12        'INVALID_AD_ATTEMPT',
13        'ORDER_NOT_FOUND',
14        'DAILY_VIDEO_LIMIT',
15        'ALREADY_CLAIMED',
16        'TOO_MANY_ORDERS',
17      ].includes(reason)
18    )
19      return;
20    const id = `night-ad-audit:${owner}:${nightDay(at)}:${kind}:${reason}`,
21      data = { day: nightDay(at), kind, reason, count: 1, lastAt: at };
22    await tx.execute(
23      sql`insert into runtime.competition_matches(id,board,data,created_at) values(${id},'night-
overwatch:private-ad-audit-v1',${JSON.stringify(data)}::jsonb,${at}) on conflict(id) do update set
data=jsonb_set(jsonb_set(runtime.competition_matches.data,'{count}',to_jsonb((runtime.competition_matches.data-
>>'count')::bigint+1)),'{lastAt}',to_jsonb(${at}::bigint))`,
24    );
25  }
26  /** Skip rows already held by gameplay: an asset read must never wait room -> identity in reverse. */
27  export async function reconcileNightLoadouts(
28    tx: Tx,
29    owner: string,
30    at: number,
31    rules: ProgressionRules,
32    versions: { solo: string; duel: string },
33    policy?: { state: string; killEpoch: number },
34  ) {
35    const epoch = policy?.killEpoch ?? 0,
36      disabled = policy?.state === 'disabled';
37    const runs = await tx.execute(
38      sql`select id,data from runtime.competition_matches where data->>'kind'='ranked-run-v1' and data-
>>'game'='night-overwatch' and data->>'playerId'=${owner} and data->>'status'='active' and data->'loadout' is not
null and data->'settlement' is null and (data->>'version'<>${versions.solo} or coalesce((data-
>>'killEpoch')::bigint,0)<>${epoch} or ${disabled}) order by id limit 20 for update skip locked`,
39    );
40    const rooms = await tx.execute(
41      sql`select code,data from runtime.competition_rooms where data->>'game'='night-overwatch' and data-
>>'status'='playing' and data->>'loadoutBattle' is not null and data->'settledAt' is null and exists(select 1 from
jsonb_array_elements(data->'players') p where p->>'id'=${owner}) and (data->>'version'<>${versions.duel} or
coalesce((data->>'killEpoch')::bigint,0)<>${epoch} or ${disabled}) order by code limit 20 for update skip locked`,
42    );
43    const refunds = [
44      ...runs.map((row) => ({
45        owner,
46        battle: `run:${(row.data as RankedRun).id}`,
47      })),
48      ...rooms.flatMap((row) => {
49        const room = row.data as Room;
50        return room.players.map((p) => ({
```

night-assets.ts:1021 — night-assets.ts:1071（排版副本行号）

完整源程序第 537 页 共 549 页

```
          owner: p.id,
          battle: room.loadoutBattle!,
        }));
      }),
    ].sort(
      (a, b) =>
        a.owner.localeCompare(b.owner) || a.battle.localeCompare(b.battle),
    );
    for (const item of refunds)
      await refundNightLoadout(tx, item.owner, at, item.battle, rules);
    for (const row of runs)
      await tx.execute(
        sql`update runtime.competition_matches set data=jsonb_set(data,'{status}','"finished"'::jsonb) where
id=${row.id}`,
      );
    for (const row of rooms)
      await tx.execute(
        sql`update runtime.competition_rooms set data=jsonb_set(data,'{status}','"abandoned"'::jsonb) where
code=${row.code}`,
      );
  }
  export async function reserveNightLoadout(
    tx: Tx,
    owner: string,
    at: number,
    battle: string,
    mode: 'solo' | 'duel',
    rules: ProgressionRules,
  ) {
    const w = await locked(tx, owner, at, rules),
      prior = w.reservations[battle];
    if (prior && ['reserved', 'consumed'].includes(prior.status))
      return prior.snapshot;
    const loadout = w.loadouts[mode],
      effects: string[] = [],
      items: string[] = [],
      skipped: string[] = [];
    for (const id of [
      ...loadout.mods,
      ...[loadout.weapon, loadout.skill].filter((x): x is string => !!x),
    ]) {
      const p = rules.catalog.find((p) => p.id === id)!;
      if (w.permanent.includes(p.effect) || (w.active[p.effect] ?? 0) > at)
        effects.push(p.effect);
      else if (p.term === 'match' && available(w, id) > 0) {
        effects.push(p.effect);
        items.push(id);
      } else skipped.push(id);
    }
    const snapshot: BattleLoadout = {
      version: rules.version,
      level: rules.levelProgress(w.xp).level,
```

night-assets.ts:1072 — night-assets.ts:1121（排版副本行号）

```
      effects,
      skipped,
    };
    w.reservations[battle] = {
      snapshot,
      items,
      status: 'reserved',
      expiresAt: at + 24 * 3600000,
    };
    await save(tx, owner, w);
    return snapshot;
  }
  export async function consumeNightLoadout(
    tx: Tx,
    owner: string,
    at: number,
    battle: string,
    rules: ProgressionRules,
    release = false,
  ) {
    const w = await locked(tx, owner, at, rules),
      r = w.reservations[battle];
    if (!release && (!r || r.status === 'released' || r.status === 'refunded'))
      fail('LOADOUT_RESERVATION_EXPIRED');
    if (!r || r.status !== 'reserved') {
      // prepare() may have released an expired reservation in memory. Maintenance
      // must persist that release even when there is no consumption left to perform.
      if (release && r?.status === 'released') await save(tx, owner, w);
      return;
    }
    if (release) r.status = 'released';
    else {
      r.consumedItems = r.items.filter(
        (id) =>
          !w.permanent.includes(rules.catalog.find((p) => p.id === id)!.effect),
      );
      for (const id of r.consumedItems) stock(w, id, -1);
      r.status = 'consumed';
    }
    await save(tx, owner, w);
  }
  /** Only server invalidation calls this; a normal quit never refunds committed inventory. */
  export async function refundNightLoadout(
    tx: Tx,
    owner: string,
    at: number,
    battle: string,
    rules: ProgressionRules,
  ) {
    const w = await locked(tx, owner, at, rules),
```

night-assets.ts:1122 — night-assets.ts:1171（排版副本行号）

```
    r = w.reservations[battle];
  if (!r || r.status === 'released' || r.status === 'refunded') {
    if (r?.status === 'released') await save(tx, owner, w);
    return;
  }
  if (r.status === 'consumed')
    for (const id of r.consumedItems ?? r.items) stock(w, id, 1);
  r.status = 'refunded';
  await save(tx, owner, w);
}
export async function rewardNightBattle(
  tx: Tx,
  owner: string,
  at: number,
  battle: string,
  rules: ProgressionRules,
  input: {
    mode: 'solo' | 'duel' | 'bot';
    outcome: 'win' | 'loss' | 'draw';
    durationMs: number;
    valid: boolean;
    levelId?: string;
  },
) {
  const w = await locked(tx, owner, at, rules),
    id = eventId(owner, `battle:${battle}`);
  const prior = await tx.execute(
    sql`select data from runtime.competition_matches where id=${id}`,
  );
  if (prior[0]) return (prior[0].data as { receipt: BattleReward }).receipt;
  const valid = input.valid && input.durationMs >= 10000;
  const reward: Reward = valid
    ? input.mode === 'solo'
      ? {
          coins: input.outcome === 'win' ? 180 : 40,
          xp: input.outcome === 'win' ? 120 : 35,
        }
      : input.mode === 'duel'
        ? {
            coins:
              input.outcome === 'win'
                ? 200
                : input.outcome === 'draw'
                  ? 100
                  : 60,
            xp: input.outcome === 'win' ? 140 : 70,
          }
        : {
            coins: input.outcome === 'win' ? 100 : 30,
            xp: input.outcome === 'win' ? 80 : 25,
```

night-assets.ts:1172 — night-assets.ts:1221（排版副本行号）

```
          }
      : { coins: 0, xp: 0 };
    add(w, reward);
    if (valid)
      progress(w, nightDay(at), [
        'complete',
        input.mode === 'solo' ? 'solo' : 'duel',
        ...(input.outcome === 'win' ? ['win'] : []),
      ]);
    prepare(w, owner, at, rules);
    await save(tx, owner, w);
    const receipt: BattleReward = {
      ...reward,
      totalCoins: w.coins,
      totalXp: w.xp,
      level: rules.levelProgress(w.xp).level,
      status: 'credited',
    };
    await tx.execute(
      sql`insert into runtime.competition_matches(id,board,data,created_at)
values(${id},${board},${JSON.stringify({ receipt, input, at })}::jsonb,${at})`,
    );
    return receipt;
  }
  import { createHash } from 'node:crypto';
  import { setTimeout } from 'node:timers/promises';
  import { sql } from 'drizzle-orm';
  import { z } from 'zod';
  import type { openDatabase } from '@coffeeeeffoc/service-kit';
  import { assertResourceAuth, playerIdOf, type PlayerAuth } from './auth.js';
  import {
    assertGameControl,
    gamePolicy,
    type CompetitionControl,
  } from './control.js';
  import { publicProfile } from './profile.js';
  import { CompetitionError, type Room } from './types.js';
  type Database = ReturnType<typeof openDatabase>['db'];
  const hash = (value: unknown) =>
    createHash('sha256').update(JSON.stringify(value)).digest('hex');
  const seq = z
    .string()
    .regex(/^[1-9][0-9]{0,18}$/)
    .refine((value) => BigInt(value) <= 9223372036854775807n);
  const text = z
    .string()
    .max(1000)
    .transform((value) => value.normalize('NFC').trim())
    .refine(
      (value) =>
        Array.from(value).length <= 120 && !/[\p{Cc}\p{Cf}]/u.test(value),
```

night-assets.ts:1222 — night-chat.ts:28（排版副本行号）

```
  );
export const chatSendSchema = z
  .object({
    requestId: z.uuid(),
    text,
    roomCode: z
      .string()
      .regex(/^(?:[0-9]{6}|[A-F0-9]{12})$/)
      .optional(),
  })
  .strict()
  .refine((value) => !!value.text || !!value.roomCode);
export const chatReportSchema = z
  .object({ id: seq, reason: z.string().trim().min(1).max(120) })
  .strict();
export const chatModerateSchema = z.discriminatedUnion('kind', [
  z.object({ kind: z.literal('remove'), id: seq }).strict(),
  z
    .object({
      kind: z.literal('mute'),
      playerId: z.uuid(),
      minutes: z.number().int().min(0).max(43200),
    })
    .strict(),
]);
/** A single channel lock gives cursors committed order across processes. No transaction is held while waiting.
*/
export function createNightChat(
  db: Database,
  now: () => number,
  control?: CompetitionControl,
  version?: string,
) {
  const access = async (auth: PlayerAuth) =>
    db.transaction(async (tx) => {
      const policy = await gamePolicy(control, tx, 'night-overwatch');
      await assertResourceAuth(tx, auth, 'night-overwatch', now());
      assertGameControl(policy, policy?.killEpoch, true);
      return policy;
    });
  const snapshot = async (epoch?: number) => {
    const rows =
      await db.execute(sql`select m.seq,m.body,m.room,m.created_at,p.id,p.display_name,p.profile,r.data as
current_room
      from runtime.night_chat_messages m join runtime.competition_players p on p.id=m.player_id
      left join runtime.competition_rooms r on r.code=m.room->>'code'
      where not m.removed and m.created_at>${now() - 86400000} order by m.seq desc limit 100`);
    const messages = [...rows].reverse().map((row) => {
      const ref = row.room as { code: string; createdAt: number } | null;
      let room:
        | { code: string; createdAt: number; canJoin: boolean }
        | undefined;
```

night-chat.ts:29 — night-chat.ts:79（排版副本行号）

```
01        if (ref) {
02          const data = row.current_room as Room | undefined;
03          room = {
04            ...ref,
05            canJoin:
06              !!data &&
07              data.game === 'night-overwatch' &&
08              data.version === version &&
09              (epoch === undefined || (data.killEpoch ?? 0) === epoch) &&
10              data.createdAt === ref.createdAt &&
11              data.visibility !== 'private' &&
12              data.status === 'waiting' &&
13              data.players.length < 2 &&
14              data.deadline > now(),
15          };
16        }
17        const profile = publicProfile(row);
18        return {
19          id: String(row.seq),
20          playerId: String(row.id),
21          nickname: profile.nickname,
22          text: String(row.body),
23          at: Number(row.created_at),
24          ...(room ? { room } : {}),
25        };
26      });
27      return { messages, cursor: hash(messages), reset: true, serverNow: now() };
28    };
29    return {
30      async list(
31        auth: PlayerAuth,
32        cursor?: string,
33        waitMs = 0,
34        cancelled = () => false,
35      ) {
36        const policy = await access(auth),
37          until = Date.now() + Math.min(5000, Math.max(0, waitMs));
38        let result = await snapshot(policy?.killEpoch);
39        while (cursor === result.cursor && Date.now() < until && !cancelled()) {
40          await setTimeout(Math.min(500, until - Date.now()));
41          result = await snapshot(policy?.killEpoch);
42        }
43        const latest = await access(auth);
44        return latest?.killEpoch === policy?.killEpoch
45          ? result
46          : snapshot(latest?.killEpoch);
47      },
48      async send(auth: PlayerAuth, value: unknown) {
49        const input = chatSendSchema.parse(value),
50          owner = playerIdOf(auth),
```

night-chat.ts:80 — night-chat.ts:129（排版副本行号）

```
          signature = hash(input);
        await db.transaction(async (tx) => {
          const policy = await gamePolicy(control, tx, 'night-overwatch');
          await tx.execute(sql`select pg_advisory_xact_lock(7142902)`);
          await assertResourceAuth(tx, auth, 'night-overwatch', now());
          assertGameControl(policy, policy?.killEpoch, true);
          const previous = await tx.execute(
            sql`select signature from runtime.night_chat_messages where player_id=${owner} and
request_id=${input.requestId}`,
          );
          if (previous[0]) {
            if (previous[0].signature !== signature)
              throw new CompetitionError('REQUEST_ID_CONFLICT');
            return;
          }
          await tx.execute(
            sql`insert into runtime.night_chat_limits(player_id) values(${owner}) on conflict do nothing`,
          );
          const limits = (
            await tx.execute(
              sql`select * from runtime.night_chat_limits where player_id=${owner} for update`,
            )
          )[0]!;
          if (Number(limits.muted_until) > now())
            throw new CompetitionError('CHAT_MUTED', 403);
          if (Number(limits.last_sent) && now() - Number(limits.last_sent) < 3000)
            throw new CompetitionError('CHAT_RATE_LIMIT', 429);
          let room: { code: string; createdAt: number } | null = null;
          if (input.roomCode) {
            const rows = await tx.execute(
              sql`select data from runtime.competition_rooms where code=${input.roomCode}`,
            );
            const current = rows[0]?.data as Room | undefined;
            if (
              !current ||
              current.game !== 'night-overwatch' ||
              current.version !== version ||
              (policy && (current.killEpoch ?? 0) !== policy.killEpoch) ||
              current.visibility === 'private' ||
              current.status !== 'waiting' ||
              current.deadline <= now() ||
              current.players.length >= 2 ||
              !current.players.some((p) => p.id === owner)
            )
              throw new CompetitionError('INVITATION_UNAVAILABLE', 422);
            room = { code: current.code, createdAt: current.createdAt };
          }
          await tx.execute(sql`insert into
runtime.night_chat_messages(request_id,player_id,signature,body,room,created_at)
            values(${input.requestId},${owner},${signature},${input.text},${room ? JSON.stringify(room) :
null}::jsonb,${now()})`);
          await tx.execute(
            sql`update runtime.night_chat_limits set last_sent=${now()} where player_id=${owner}`,
```

night-chat.ts:130 — night-chat.ts:179（排版副本行号）

```
01          );
02          // Keep request digests for safe old retries, but remove expired public text and invitations.
03          await tx.execute(sql`update runtime.night_chat_messages set removed=true,body='',room=null where not
removed and
04            (created_at<=${now() - 86400000} or seq<(select min(seq) from (select seq from
runtime.night_chat_messages order by seq desc limit 100) recent))`);
05        });
06        return snapshot((await access(auth))?.killEpoch);
07      },
08      async report(auth: PlayerAuth, value: unknown) {
09        const input = chatReportSchema.parse(value);
10        await access(auth);
11        const owner = playerIdOf(auth);
12        return db.transaction(async (tx) => {
13          await tx.execute(sql`select pg_advisory_xact_lock(7142902)`);
14          await assertResourceAuth(tx, auth, 'night-overwatch', now());
15          const old = await tx.execute(
16            sql`select 1 from runtime.night_chat_reports where reporter_id=${owner} and
message_seq=${input.id}::bigint`,
17          );
18          if (old[0]) return { reported: true };
19          const count = await tx.execute(
20            sql`select count(*)::int as n from runtime.night_chat_reports where reporter_id=${owner} and
created_at>${now() - 86400000}`,
21          );
22          if (Number(count[0]?.n) >= 20)
23            throw new CompetitionError('CHAT_RATE_LIMIT', 429);
24          const row = (
25            await tx.execute(
26              sql`select * from runtime.night_chat_messages where seq=${input.id}::bigint and not removed and
created_at>${now() - 86400000}`,
27            )
28          )[0];
29          if (!row) throw new CompetitionError('MESSAGE_NOT_FOUND', 404);
30          await tx.execute(
31            sql`insert into runtime.night_chat_reports(reporter_id,message_seq,evidence,reason,created_at)
values(${owner},${input.id}::bigint,${JSON.stringify(row)}::jsonb,${input.reason},${now()})`,
32          );
33          return { reported: true };
34        });
35      },
36      async moderate(value: unknown) {
37        const input = chatModerateSchema.parse(value);
38        return db.transaction(async (tx) => {
39          await tx.execute(sql`select pg_advisory_xact_lock(7142902)`);
40          if (input.kind === 'remove')
41            await tx.execute(
42              sql`update runtime.night_chat_messages set removed=true,body='',room=null where
seq=${input.id}::bigint`,
43            );
44          else
45            await tx.execute(
46              sql`insert into runtime.night_chat_limits(player_id,muted_until) values(${input.playerId},${now()
+ input.minutes * 60000}) on conflict(player_id) do update set muted_until=excluded.muted_until`,
47            );
48          return { applied: true };
49        });
50      },
```

night-chat.ts:180 — night-chat.ts:229（排版副本行号）

```
      async reports() {
        return {
          reports: await db.execute(
            sql`select * from runtime.night_chat_reports order by created_at desc limit 100`,
          ),
        };
      },
    };
  }
  import { sql } from 'drizzle-orm';
  import type { openDatabase } from '@coffeeeeffoc/service-kit';
  import { CompetitionError, type Room } from './types.js';
  import { rewardNightBattle, type ProgressionRules } from './night-assets.js';
  type Database = ReturnType<typeof openDatabase>['db'];
  type Tx = Parameters<Parameters<Database['transaction']>[0]>[0];
  export const nightBoard = (board: string) =>
    `night-overwatch:${board}:night-records-v1`;
  export const nightLedgerLock = (tx: Tx) =>
    tx.execute(
      sql`select pg_advisory_xact_lock(hashtext('night-overwatch-ledger'))`,
    );
  const clearsMigration = 'night-migration:highest-clear-v2';
  async function clearsMigrated(tx: Tx) {
    const rows = await tx.execute(sql`select id from runtime.competition_matches
      where id=${clearsMigration} and data->>'status'='complete'`);
    return rows.length > 0;
  }
  /** Old verified fastest times remain useful; old events never award new points. */
  export async function backfillNightClears(tx: Tx, at: number) {
    if (await clearsMigrated(tx)) return;
    await nightLedgerLock(tx);
    if (await clearsMigrated(tx)) return;
    await tx.execute(sql`insert into
runtime.competition_best(board,player_id,score,secondary,match_id,updated_at)
      select 'night-overwatch:' || split_part(board,':',2) || ':night-records-
v1',player_id,1,min(secondary),min(match_id),${at}
      from runtime.competition_results where board ~ '^night-overwatch:level-[a-z0-9-]+:night-input-replay-v[0-
9]+$' and score>0
      group by player_id,split_part(board,':',2)
      on conflict(board,player_id) do update set
secondary=least(runtime.competition_best.secondary,excluded.secondary)`);
    await refreshNightClears(tx, at);
    await tx.execute(sql`insert into runtime.competition_matches(id,board,data,created_at)
      values(${clearsMigration},${nightBoard('migration')},'{"status":"complete"}'::jsonb,${at})
      on conflict(id) do update set data=excluded.data,created_at=excluded.created_at`);
  }
  /** All players only during migration; each new clear refreshes its owner. Caller holds ledger lock. */
  export async function refreshNightClears(
    tx: Tx,
    at: number,
    playerId?: string,
  ) {
    const player = playerId ? sql`and player_id=${playerId}` : sql``;
    await tx.execute(sql`insert into
runtime.competition_best(board,player_id,score,secondary,match_id,updated_at)
```

night-chat.ts:230 — night-ledger.ts:45（排版副本行号）

完整源程序第 546 页 共 549 页

```
01      select ${nightBoard('clears')},player_id,max(substring(level from '[0-9]+$')::int),0,'night-clears:' ||
player_id,${at}
02      from (select player_id,substring(split_part(board,':',2) from 7) as level from runtime.competition_best
03        where board like 'night-overwatch:level-%:night-records-v1' and score>0 ${player}
04        union select player_id,substring(split_part(board,':',2) from 7) as level from
runtime.competition_results
05        where board like 'night-overwatch:clear-%:night-records-v1' and score>0 ${player}) cleared
06      where level in ('corridor-01','ambush-02','patrol-03') or level ~ '^operation-(00[4-9]|0[1-9][0-
9]|1[01][0-9]|120)$'
07      group by player_id
08      on conflict(board,player_id) do update set score=excluded.score,updated_at=excluded.updated_at
09      where runtime.competition_best.score<>excluded.score`);
10  }
11  export async function nightSummary(tx: Tx | Database, playerId: string) {
12    const rows = await tx.execute(sql`select count(*)::int as games,
13      count(*) filter(where data->>'outcome'='win')::int as wins,
14      count(*) filter(where data->>'outcome'='loss')::int as losses,
15      count(*) filter(where data->>'outcome'='draw')::int as draws
16      from runtime.competition_matches where board=${nightBoard('history')} and data->>'playerId'=${playerId}`);
17    const { wins, losses, draws, games } = rows[0]!;
18    return {
19      wins: Number(wins),
20      losses: Number(losses),
21      draws: Number(draws),
22      games: Number(games),
23      winRate: Number(games) ? Number(wins) / Number(games) : 0,
24    };
25  }
26  export async function nightPoints(
27    tx: Tx,
28    eventId: string,
29    playerId: string,
30    points: number,
31    at: number,
32  ) {
33    await nightLedgerLock(tx);
34    const inserted =
35      await tx.execute(sql`insert into
runtime.competition_results(match_id,board,player_id,score,secondary,created_at)
36      values(${`night-points:${eventId}`},${nightBoard('points')},${playerId},${points},0,${at}) on conflict do
nothing returning player_id`);
37    if (inserted.length)
38      await tx.execute(sql`insert into
runtime.competition_best(board,player_id,score,secondary,match_id,updated_at)
39      select ${nightBoard('points')},${playerId},coalesce(sum(score),0),0,${eventId},${at}
40      from runtime.competition_results where board=${nightBoard('points')} and player_id=${playerId}
41      on conflict(board,player_id) do update set
score=excluded.score,match_id=excluded.match_id,updated_at=excluded.updated_at`);
42    const rows = await tx.execute(
43      sql`select score from runtime.competition_best where board=${nightBoard('points')} and
player_id=${playerId}`,
44    );
45    return Number(rows[0]?.score ?? 0);
46  }
47  export async function settleNightRoom(
48    tx: Tx,
49    room: Room,
50    at: number,
```

night-ledger.ts:46 — night-ledger.ts:98（排版副本行号）

```
01    reason: 'elimination' | 'disconnect' | 'abandoned',
02    progression?: ProgressionRules,
03  ) {
04    if (room.settledAt !== undefined || !room.startedAt || at < room.startedAt)
05      return;
06    await nightLedgerLock(tx);
07    const matchId = room.id ?? room.code;
08    const human = room.players.length === 2 && room.mode !== 'bot';
09    for (const member of room.players) {
10      const outcome =
11        reason === 'abandoned'
12          ? 'loss'
13          : reason === 'disconnect'
14            ? member.takenOver
15              ? 'loss'
16              : 'win'
17            : member.result!.outcome!;
18      const points =
19        reason === 'abandoned'
20          ? 0
21          : human
22            ? { win: 120, loss: 40, draw: 60 }[outcome]
23            : { win: 40, loss: 10, draw: 20 }[outcome];
24      if (human) {
25        const opponent = room.players.find((other) => other.id !== member.id)!;
26        const record = {
27          id: matchId,
28          code: room.code,
29          playerId: member.id,
30          opponentId: opponent.id,
31          outcome,
32          reason,
33          startedAt: room.startedAt,
34          finishedAt: at,
35          durationMs: Math.max(0, at - room.startedAt),
36          points,
37        };
38        await tx.execute(sql`insert into runtime.competition_matches(id,board,data,created_at)
39          values(${`night-
history:${matchId}:${member.id}`},${nightBoard('history')},${JSON.stringify(record)}::jsonb,${at}) on conflict do
nothing`);
40      }
41      const totalPoints = await nightPoints(
42        tx,
43        `duel:${matchId}`,
44        member.id,
45        points,
46        at,
47      );
48      const summary = await nightSummary(tx, member.id);
49      if (human)
50        await tx.execute(sql`insert into
runtime.competition_best(board,player_id,score,secondary,match_id,updated_at)
```

night-ledger.ts:99 — night-ledger.ts:148（排版副本行号）

```
        values(${nightBoard('win-rate')},${member.id},${summary.winRate},0,${matchId},${at})
        on conflict(board,player_id) do update set
score=excluded.score,secondary=0,match_id=excluded.match_id,updated_at=excluded.updated_at`);
      member.settlement = {
        points,
        totalPoints,
        wins: summary.wins,
        games: summary.games,
        outcome,
        reason,
      };
      if (progression)
        Object.assign(
          member.settlement,
          await rewardNightBattle(
            tx,
            member.id,
            at,
            room.loadoutBattle ?? `duel:${matchId}`,
            progression,
            {
              mode: human ? 'duel' : 'bot',
              outcome,
              durationMs: at - room.startedAt,
              valid:
                reason === 'elimination' &&
                member.result?.participated === true &&
                !member.takenOver,
            },
          ),
        );
    }
    room.settledAt = at;
  }
  export async function nightHistory(
    tx: Tx | Database,
    playerId: string,
    cursor?: string,
  ) {
    let before: { at: number; id: string } | undefined;
    if (cursor) {
      try {
        before = JSON.parse(Buffer.from(cursor, 'base64url').toString('utf8'));
        if (
          !before ||
          !Number.isSafeInteger(before.at) ||
          before.at < 0 ||
          !/^(?:[0-9]{6}|[A-F0-9]{12}|[a-f0-9]{8}-[a-f0-9]{4}-[1-5][a-f0-9]{3}-[89ab][a-f0-9]{3}-[a-f0-
9]{12})$/.test(
            before.id,
          )
        )
```

night-ledger.ts:149 — night-ledger.ts:199（排版副本行号）

```
          throw Error();
      } catch {
        throw new CompetitionError('INVALID_CURSOR', 422);
      }
    }
    const rows =
      await tx.execute(sql`select m.data,p.id as opponent_id,p.display_name as opponent_name from
runtime.competition_matches m
      join runtime.competition_players p on p.id::text=m.data->>'opponentId'
      where m.board=${nightBoard('history')} and m.data->>'playerId'=${playerId}
      ${before ? sql`and (m.created_at,m.data->>'id') < (${before.at},${before.id})` : sql``}
      order by m.created_at desc,m.data->>'id' desc limit 21`);
    const matches = rows.slice(0, 20).map((row) => {
      const data = row.data as {
        id: string;
        code: string;
        outcome: string;
        reason: string;
        startedAt: number;
        finishedAt: number;
        durationMs: number;
        points: number;
      };
      return {
        id: data.id,
        code: data.code,
        opponent: {
          id: String(row.opponent_id),
          name: String(row.opponent_name ?? '新玩家'),
        },
        outcome: data.outcome,
        reason: data.reason,
        startedAt: data.startedAt,
        finishedAt: data.finishedAt,
        durationMs: data.durationMs,
        points: data.points,
      };
    });
    const last = matches.at(-1);
    return {
      summary: await nightSummary(tx, playerId),
      matches,
      ...(rows.length > 20 && last
        ? {
            nextCursor: Buffer.from(
              JSON.stringify({ at: last.finishedAt, id: last.id }),
            ).toString('base64url'),
          }
        : {}),
    };
  }
```

night-ledger.ts:200 — night-ledger.ts:249（排版副本行号）